2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В качестве основного языка программирования для разработки desktop-приложения «Training Reaction» выбран язык Python. Данный язык широко используется для разработки прикладного программного обеспечения, в том числе настольных приложений, благодаря своей универсальности, простоте и большому количеству доступных библиотек.

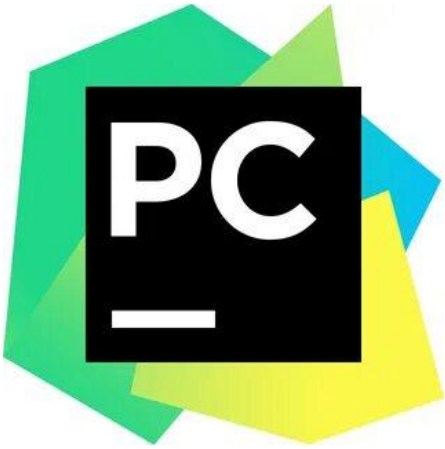

Рисунок 2.1 – Среда разработки PyCharm

Язык Python позволяет быстро создавать рабочие прототипы приложений и постепенно расширять их функциональные возможности. Кроме того, Python поддерживает работу с графическими интерфейсами, обработку пользовательского ввода и точное измерение времени, что делает его удобным для реализации приложений по тренировке реакции.

Достоинства языка Python:

- Простой и понятный синтаксис;
- Высокая скорость разработки программ;
- Возможность создания полноценных desktop-приложений;
- Кроссплатформенность;
- Большое количество готовых библиотек;
- Активное сообщество и хорошая документация.

Недостатки языка Python**:**

- Более низкая производительность по сравнению с некоторыми другими языками;
- Интерпретируемый характер выполнения программ;
- Для распространения готового приложения могут потребоваться дополнительные инструменты.

Несмотря на перечисленные недостатки, Python хорошо подходит для разработки приложения «Training Reaction», так как проект ориентирован на прикладное использование, простоту и удобство работы пользователя.

В качестве среды разработки используется PyCharm. Это специализированная среда разработки для языка Python, которая предоставляет удобные инструменты для написания, редактирования и отладки программного кода. Он автоматически проверяет код на ошибки, помогает с подсказками при написании программ, облегчает навигацию по проекту и позволяет удобно управлять файлами и библиотеками. Использование данной среды разработки повышает качество кода и ускоряет процесс разработки приложения.

Для реализации desktop-приложения «Training Reaction» планируется использование стандартных и сторонних библиотек языка Python. На текущем этапе разработки перечень библиотек не является окончательно сформированным и будет уточняться по мере реализации функционала приложения.

В качестве основы для создания графического пользовательского интерфейса рассматривается использование одной из библиотек для разработки графических приложений на Python, таких как Tkinter или PyQt. Выбор данных библиотек обусловлен их устойчивостью, широким распространением и возможностью создания полноценных оконных приложений.

Использование специализированных библиотек позволит:

- реализовать удобный и понятный пользовательский интерфейс;
- обеспечить точную фиксацию времени реакции пользователя;
- организовать обработку пользовательского ввода с клавиатуры и мыши;
- хранить и анализировать результаты тренировок.

Конкретный перечень библиотек, а также обоснование их использования будет сформирован и дополнен на последующих этапах разработки программного обеспечения.

Скриншот работы с IDE PyCharm приведен на рисунке 2.2

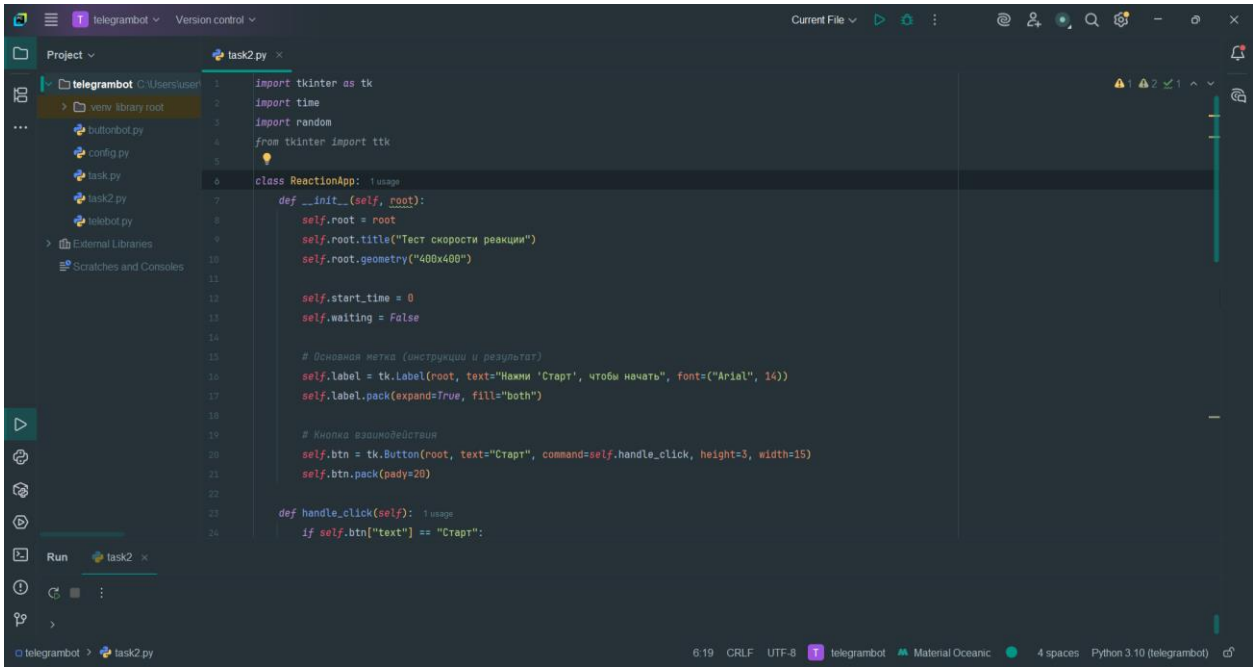

Рисунок 2.2 – Интерфейс PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub.

Имя пользователя на GitHub – bobik3223. Ссылка на профиль – https://github.com/bobik3223. Ссылка на репозиторий – https://github.com/bobik3223/UP.

Скриншот работы с IDE PyCharm приведен на рисунке 2.3

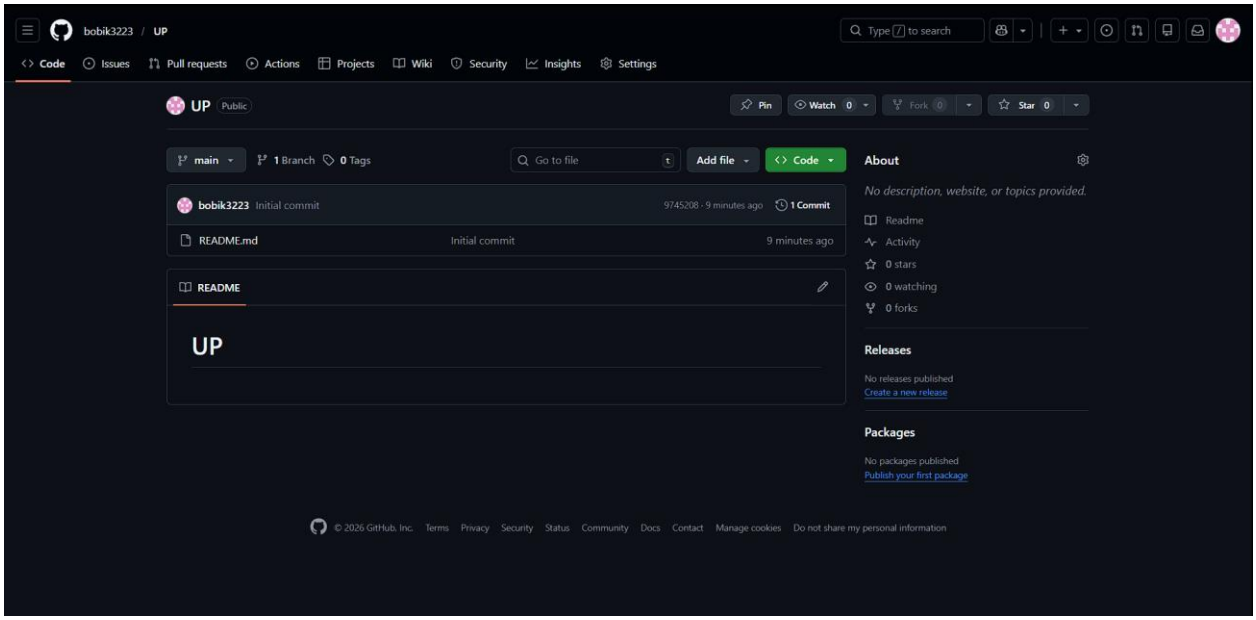

Рисунок 2.3 – Работа с GitHub

4 ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Реакция человека является одной из важнейших характеристик его нервной системы и отражает способность быстро воспринимать внешние сигналы и отвечать на них определённым действием. В психологии под реакцией понимается временной интервал между моментом появления раздражителя и началом ответного действия человека. [1]

Скорость реакции напрямую связана с работой органов чувств, центральной нервной системы и двигательного аппарата [1]. Чем быстрее человек воспринимает сигнал и выполняет ответное действие, тем выше считается уровень его сенсомоторной подготовленности. Данная характеристика играет важную роль в спорте, профессиональной деятельности, управлении техникой и повседневной жизни [2].

В учебной и научной литературе выделяют несколько основных видов реакции человека: сенсомоторную реакцию, простую реакцию и реакцию выбора.

В рамках данного проекта основное внимание уделяется сенсомоторной реакции, так как именно она наиболее наглядно поддаётся измерению и тренировке с помощью компьютерных программ.

Сенсомоторная реакция представляет собой ответное движение человека на внешний сигнал, чаще всего зрительный или слуховой. Примером сенсомоторной реакции является нажатие клавиши клавиатуры или кнопки мыши при появлении сигнала на экране. Данный вид реакции широко используется при тестировании быстроты реакции, так как позволяет с высокой точностью измерять время отклика пользователя в миллисекундах.

Использование компьютерных программ для тренировки реакции позволяет обеспечить точную фиксацию времени отклика, повторяемость упражнений и возможность анализа результатов тренировок. Такие

программные средства применяются как в учебных целях, так и для самотренировки пользователей [3].

В данной работе рассматривается разработка desktop-приложения «Training Reaction», предназначенного для тренировки реакции человека. Приложение ориентировано на тренировочный и прикладной характер использования.

В последние годы наблюдается устойчивый рост интереса к онлайн-инструментам для оценки когнитивных способностей, таких как память, внимание и скорость реакции. Это объясняется удобством доступа, достаточно браузера и подключения к интернету [5]. Согласно данным опроса, в 2024 году среди учителей государственных школ, 48% учителей заявили, что большинство учеников показывают удовлетворительную или плохую успеваемость; треть – хорошую, и только 17% – отличную или очень хорошую. Оценка и развитие когнитивных навыков могут существенно повлиять на результаты обучения и общее развитие учащихся [6].

4.1.1 Анализ проблемы

На сегодняшний день существует большое количество программ и онлайн-сервисов для измерения скорости реакции. Однако большинство из них имеют ряд существенных недостатков.

Во-первых, значительная часть подобных решений реализована в виде веб-сервисов или мобильных приложений. Использование браузера и сенсорного ввода снижает точность измерений и делает такие решения менее подходящими для тренировок, что отмечается в работах, посвящённых психомоторной организации и сенсомоторным реакциям человека [4].

Во-вторых, многие существующие программы ориентированы на развлекательный формат и используют игровые элементы, что отвлекает пользователя от основной цели – тренировки реакции.

В-третьих, большинство приложений не предоставляет возможности хранения статистики и анализа результатов, таких как вычисление среднего времени реакции, отслеживание прогресса пользователя и сравнение результатов за различные периоды времени.

Таким образом, существует необходимость в разработке desktop-приложения, ориентированного на тренировку реакции человека с простым интерфейсом и возможностью регулярного использования.

4.1.2 Обзор аналогов

Анализ существующих решений в области тренировки и измерения скорости реакции показывает, что на сегодняшний день количество специализированного программного обеспечения для настольных компьютеров ограничено. Большинство доступных решений представлено в виде веб-сервисов, функционирующих через браузер, и ориентировано преимущественно на разовое измерение времени реакции, а не на систематическую тренировку.

В связи с этим для анализа были рассмотрены как веб-аналоги, так и отдельный программный продукт, частично реализующие функции тренировки реакции.

ReactionTimeTest [5] – это минималистичный веб-тест, предназначенный для измерения времени реакции на смену визуального сигнала. Пользователю предлагается дождаться изменения цвета элемента интерфейса и выполнить клик мышью. Результат отображается сразу после выполнения действия.

Сервис отличается предельно простой реализацией и не содержит дополнительных элементов, что делает его удобным для быстрого измерения реакции, но ограничивает функциональные возможности.

Достоинства ReactionTimeTest:

- Доступность через браузер без установки программ;
- Возможность быстрого прохождения теста;
- Минималистичный интерфейс без отвлекающих элементов;
- Наглядное представление результатов;
- Возможность повторного прохождения теста без ограничений.

Недостатки:

- Ограничение одним тестовым сценарием;
- Отсутствие статистики и истории измерений;
- Невозможность настройки условий тестирования;
- Не предназначен для регулярного использования и тренировки;
- Слишком маленький функционал.

Примеры работ с веб-сервисом ReactionTimeTest представлены на рисунках 4.1-4.2

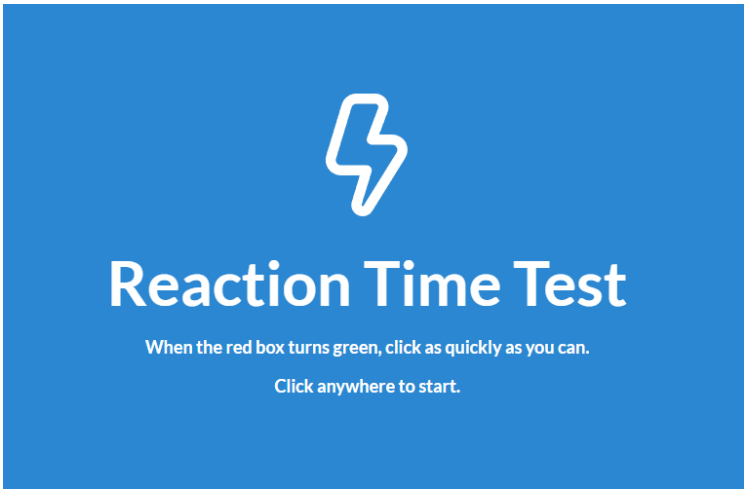

Рисунок 4.1 –Структура начального окна веб-сервиса ReactionTimeTest

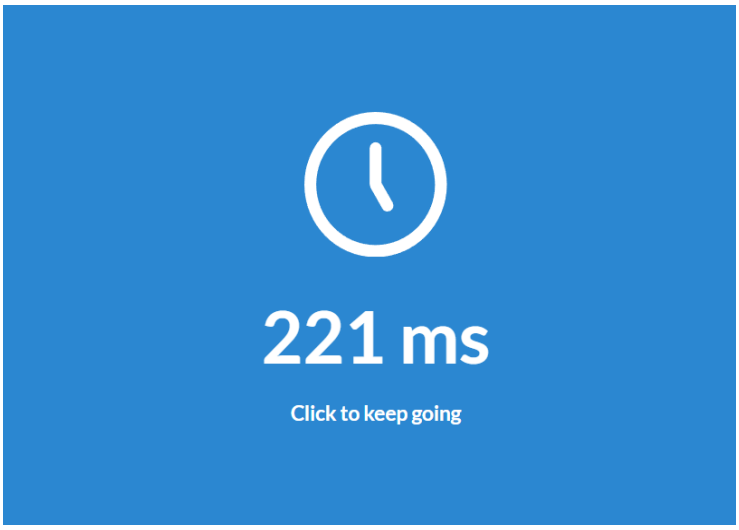

Рисунок 4.2 – Зафиксированный результат веб-сервиса ReactionTimeTest

Следующий аналог – Human Benchmark [6] представляет собой веб-сервис, предназначенный для тестирования различных когнитивных показателей человека, включая скорость реакции. Пользователю предлагается выполнить простое задание: нажать кнопку как можно быстрее после появления визуального сигнала. Результат измеряется в миллисекундах и отображается на экране.

Достоинства Human Benchmark:

- Простота использования и минималистичный интерфейс;
- Быстрый доступ к тренировке реакции;
- Наглядное отображение результата измерения.

Недостатки Human Benchmark:

- Веб-реализация, ограниченная точность измерений;
- Отсутствие возможностей для полноценной тренировки реакции;
- Единственный фиксированный режим простой реакции
- Зависимость от браузера и интернет-соединения.

Примеры работ с веб-сервисом ReactionTimeTest представлены на рисунках 4.3-4.4

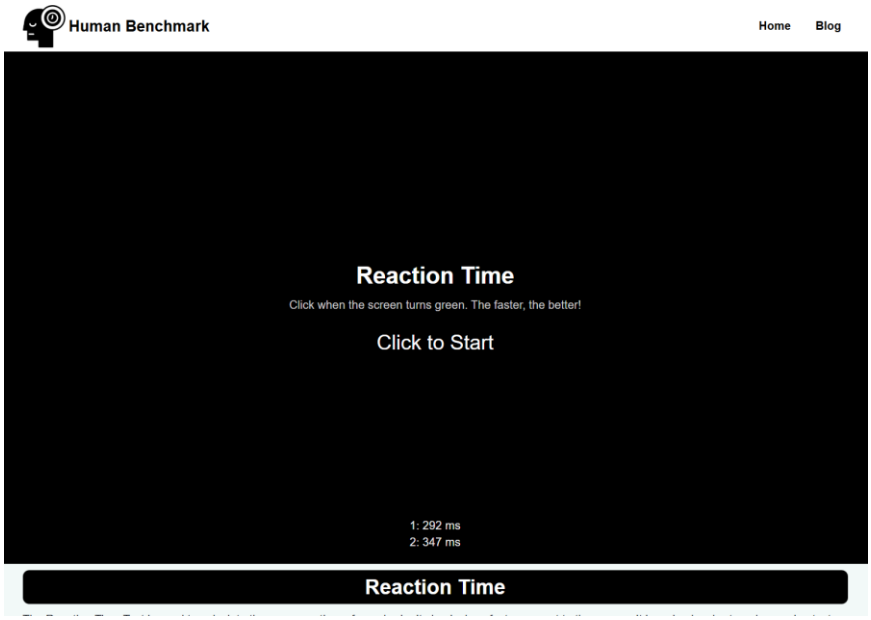

Рисунок 4.3 –Структура начального окна веб-сервиса Human Benchmark

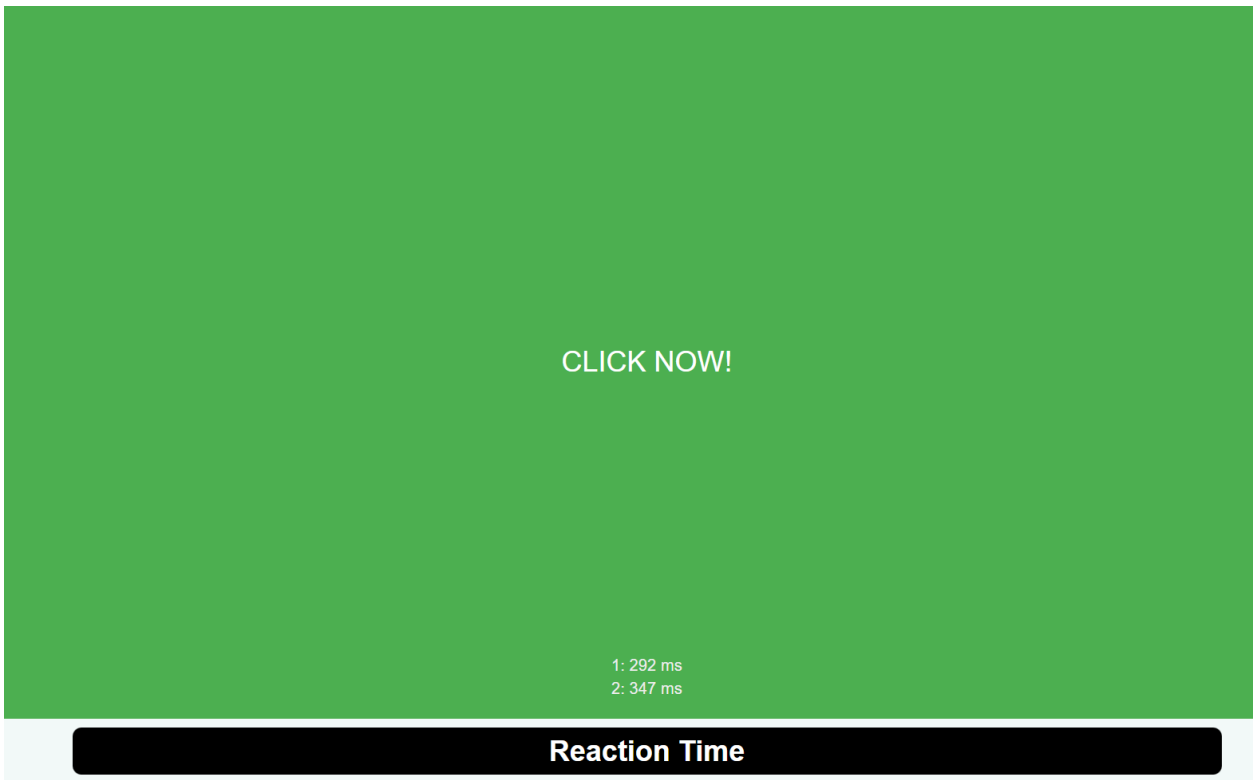

Рисунок 4.4 –Процесс тренировки реакции в веб-сервисе Human Benchmark

Аналог Reaction Timer – это программа, которая представляет собой утилиту для измерения времени простой зрительно-моторной реакции. Принцип работы основан на классической методике: после нажатия кнопки "Start Timer" следует случайная задержка, после чего в интерфейсе возникает стимул для ответа. Пользователь должен как можно быстрее отреагировать на стимул с помощью нажатия клавиши.

Достоинства Reaction Timer:

- Не требует браузера и интернет-соединения, что потенциально повышает стабильность и точность работы по сравнению с веб-аналогами;

- Минималистичный интерфейс позволяет быстро приступить к измерению без отвлекающих факторов.

Недостатки Reaction Timer:

- Крайне ограниченный функционал. Реализован лишь один базовый тест на простую реакцию, что недостаточно для комплексной тренировки;

- Внешний вид соответствует программам ранних версий Windows, что снижает эргономику и восприятие;

- Подобные утилиты часто являются заброшенным ПО, что означает потенциальные проблемы с совместимостью на новых версиях ОС и отсутствие исправлений ошибок.

Примеры работ с приложением Reaction Timer представлены на рисунках 4.5-4.6

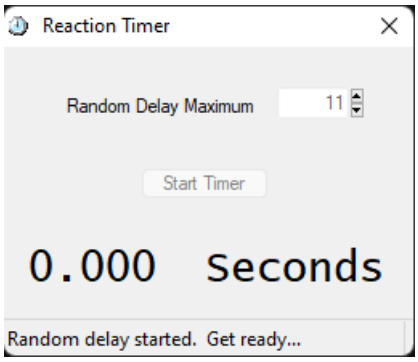

Рисунок 4.5 – Ожидание запуска тренировки для проверки реакции

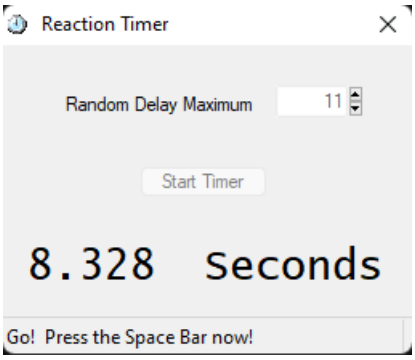

Рисунок 4.5 – Запуск тренировки для проверки реакции

Таким образом, указанные ограничения свидетельствуют о необходимости разработки desktop-приложения, способного обеспечить более точные и стабильные измерения времени реакции, не зависящие от особенностей браузера и сетевого окружения, а также более гибкого функционала и количества различных режимов тренировки.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

Алгоритм работы модуля тренировки простой сенсомоторной реакции представлен на рисунках 4.2.1. – 4.2.3

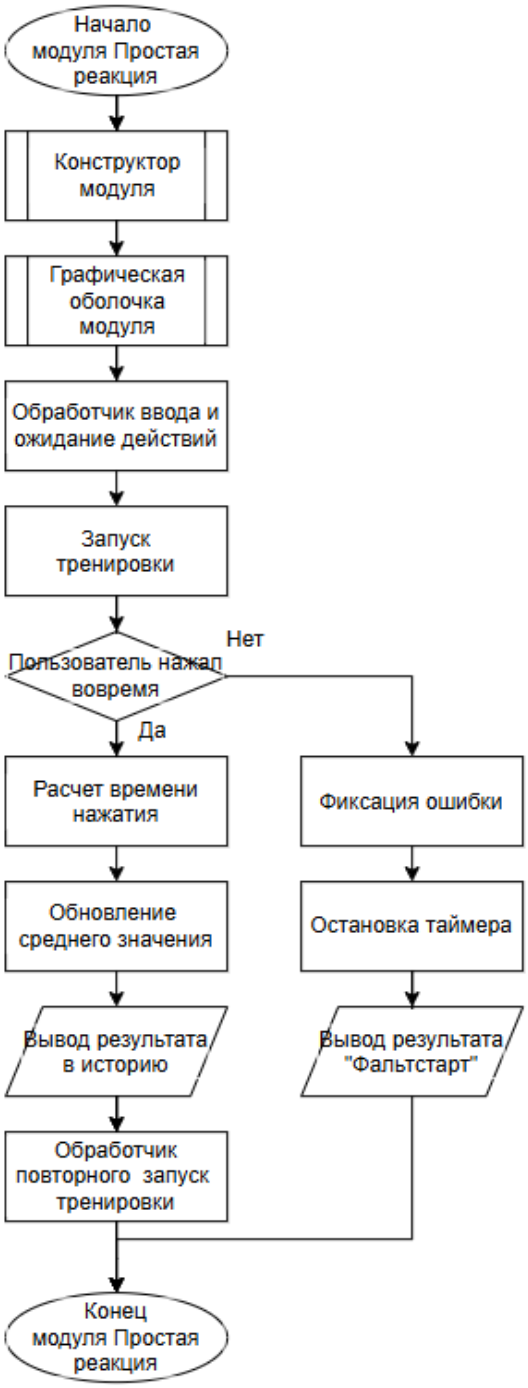

Рисунок 4.2.1 – Алгоритм работы модуля тренировки простой реакции

Рисунок 4.2.2 – Алгоритм работы конструктора модуля тренировки простой реакции

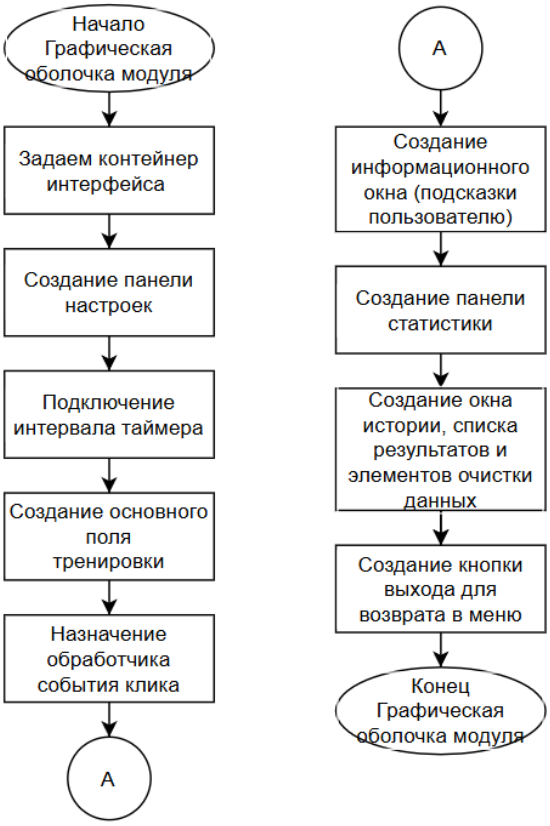

Рисунок 4.2.3 – Алгоритм работы графического модуля тренировки простой реакции

Алгоритм работы модуля «Простая сенсомоторная реакция» начинается с этапа инициализации, включающего запуск конструктора и формирование графической оболочки интерфейса. После перехода в режим ожидания и запуска тренировки система активирует программный таймер. В зависимости от действий испытуемого логика разделяется на два сценария: при своевременном отклике происходит прецизионный расчет времени нажатия, обновление статистических данных и фиксация результата в истории сессии. В случае преждевременного воздействия (фальстарта) алгоритм фиксирует ошибку, принудительно останавливает хронометраж и выводит уведомление. Цикл завершается работой обработчика повторного запуска, который подготавливает систему к следующей экспозиции стимула до полной остановки теста оператором.

Алгоритм работы модуля «Реакции на выбор» и представлен на рисунках 4.2.4. – 4.2.6

Алгоритм функционирования модуля «Реакции на выбор» базируется на последовательной инициализации компонентов конструктора и графической оболочки, после чего система переходит в режим ожидания действий пользователя через обработчик ввода. Ключевым отличием данного алгоритма является этап смены объектов появления, вносящий элемент неопределенности и требующий от испытуемого селективного внимания. В процессе работы логический блок анализирует корректность выбора: при нажатии на правильный объект выполняется прецизионный расчет времени реакции, статистическое обновление данных и фиксация результата в протоколе. В случае выбора ошибочного объекта или преждевременного воздействия срабатывает блок фиксации ошибки с последующей остановкой таймера и выводом уведомления. Завершается цикл работой обработчика повторного запуска, обеспечивающего непрерывность тренировочного процесса до команды прекращения теста.

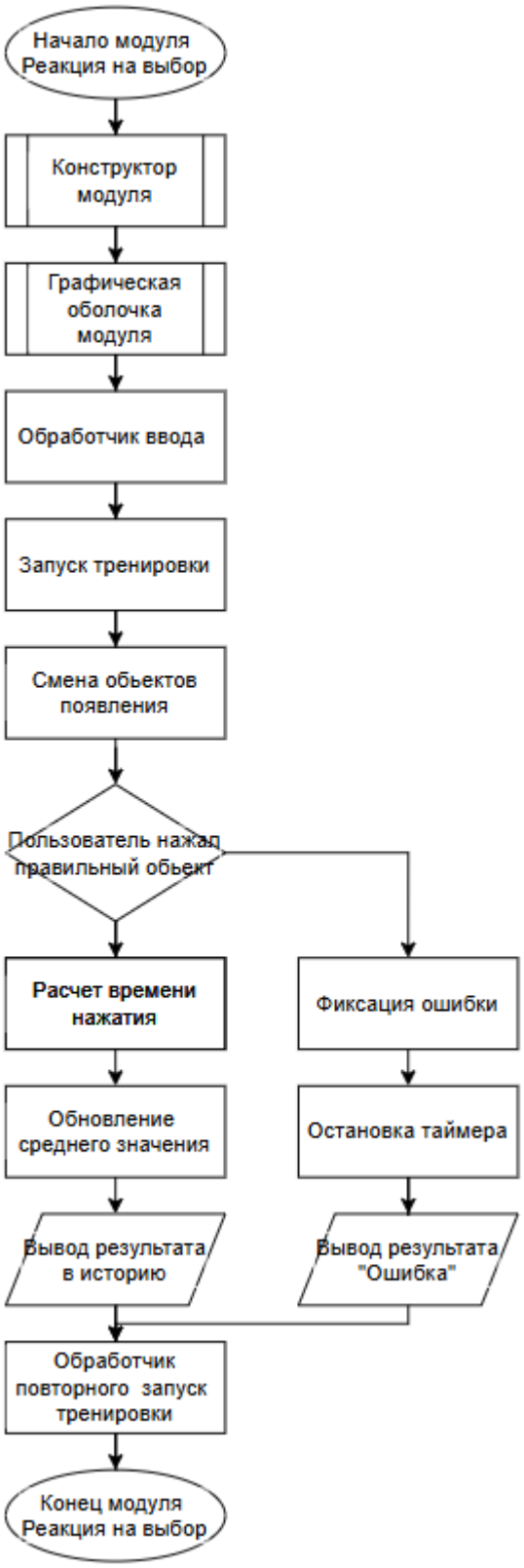

Рисунок 4.2.4 – Алгоритм работы модуля «Реакции на выбор»

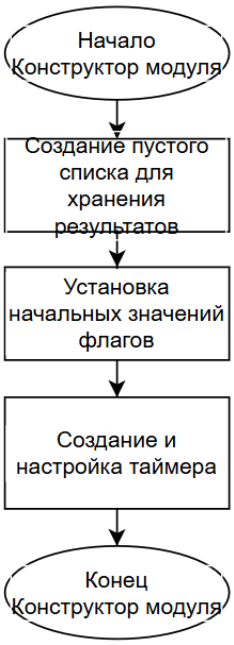

Рисунок 4.2.5 – Алгоритм работы конструктора модуля «Реакции на выбор»

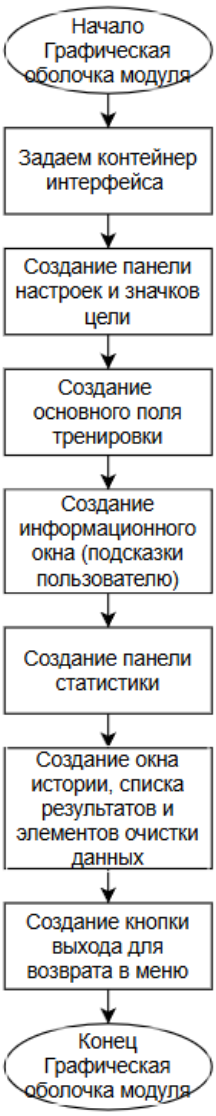

Рисунок 4.2.6 – Алгоритм работы графического модуля «Реакции на выбор»

Алгоритм работы модуля «Координация и слежение» и представлен на рисунках 4.2.7. – 4.2.9

Рисунок 4.2.7 – Алгоритм работы модуля «Координация и слежение»

Рисунок 4.2.8 – Алгоритм работы конструктора модуля «Координация и слежение»

Рисунок 4.2.9 – Алгоритм работы графического модуля «Координация и слежение»

Алгоритм функционирования модуля «Координация и слежение» (Визуально-моторная координация) инициируется через конструктор и графическую оболочку, подготавливая программную среду к работе. После запуска тренировочного цикла система выполняет динамический расчет пауз между появлениями целевых объектов, что исключает ритмическое привыкание испытуемого. При успешном попадании производится расчет времени взаимодействия, обновление средних значений скорости реакции и автоматический вывод данных в историю сессии. Завершается алгоритм работой обработчика повторного запуска, который инициирует новую итерацию появления объекта до момента принудительного завершения теста оператором.

Алгоритм работы модуля «Реакция на сложный выбор»представлен на рисунках 4.2.10. – 4.2.12

Алгоритм функционирования модуля «Реакция на сложный выбор» инициируется через конструктор и графическую оболочку, подготавливая переменные для регистрации двусторонних стимулов. После запуска сессии система активирует таймер задержки, по завершении которой происходит случайный выбор вектора воздействия (левая или правая зона) и визуальная индикация соответствующего сигнала. Ключевым этапом алгоритма является работа диспетчера аппаратного отклика, который классифицирует входящий сигнал: при совпадении нажатой клавиши с вектором стимула производится расчет времени реакции и обновление статистики; при несоответствии или фальстарте активируется блок фиксации ошибки с визуальной блокировкой зон. После обработки результата система задействует таймер паузы для стабилизации состояния испытуемого и через обработчик повторного запуска переходит к следующей итерации цикла.

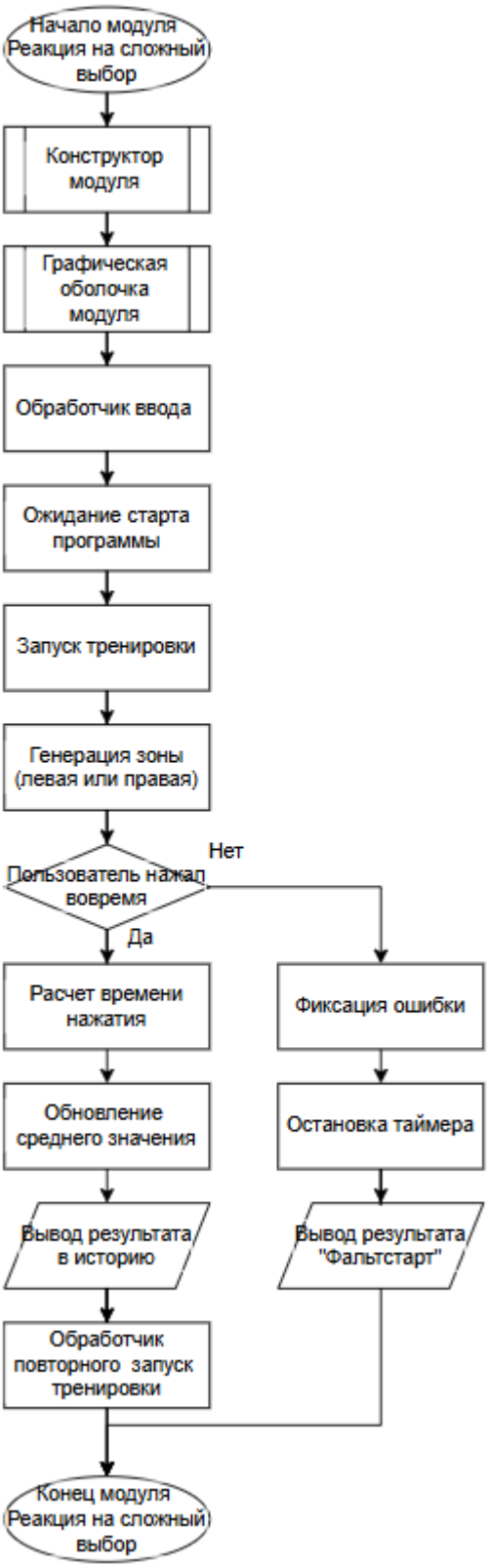

Рисунок 4.2.10 – Алгоритм работы модуля «Реакция на сложный выбор»

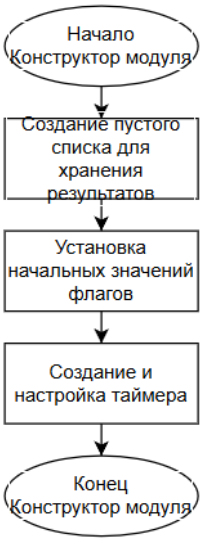

Рисунок 4.2.11 – Алгоритм работы конструктора модуля «Реакция на сложный выбор»

Рисунок 4.2.12 – Алгоритм работы графического модуля «Реакция на сложный выбор»

4.2.2 Разрабока структурных решений

Модульная структура приложения приведена на рисунке 4.2.13

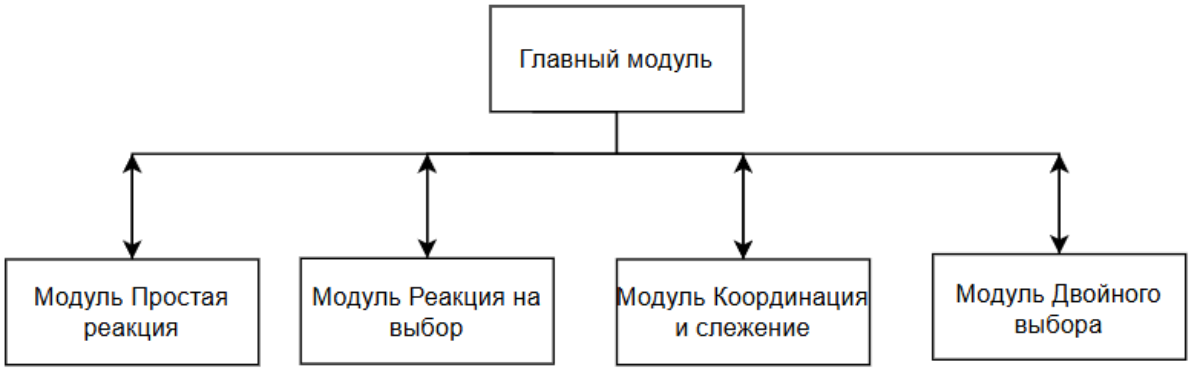

Рисунок 4.2.13 – Модульная структура приложения

4.2.3 Проектирование интерфейса

Макет главного окна приложения представлена на рисунке 4.2.14

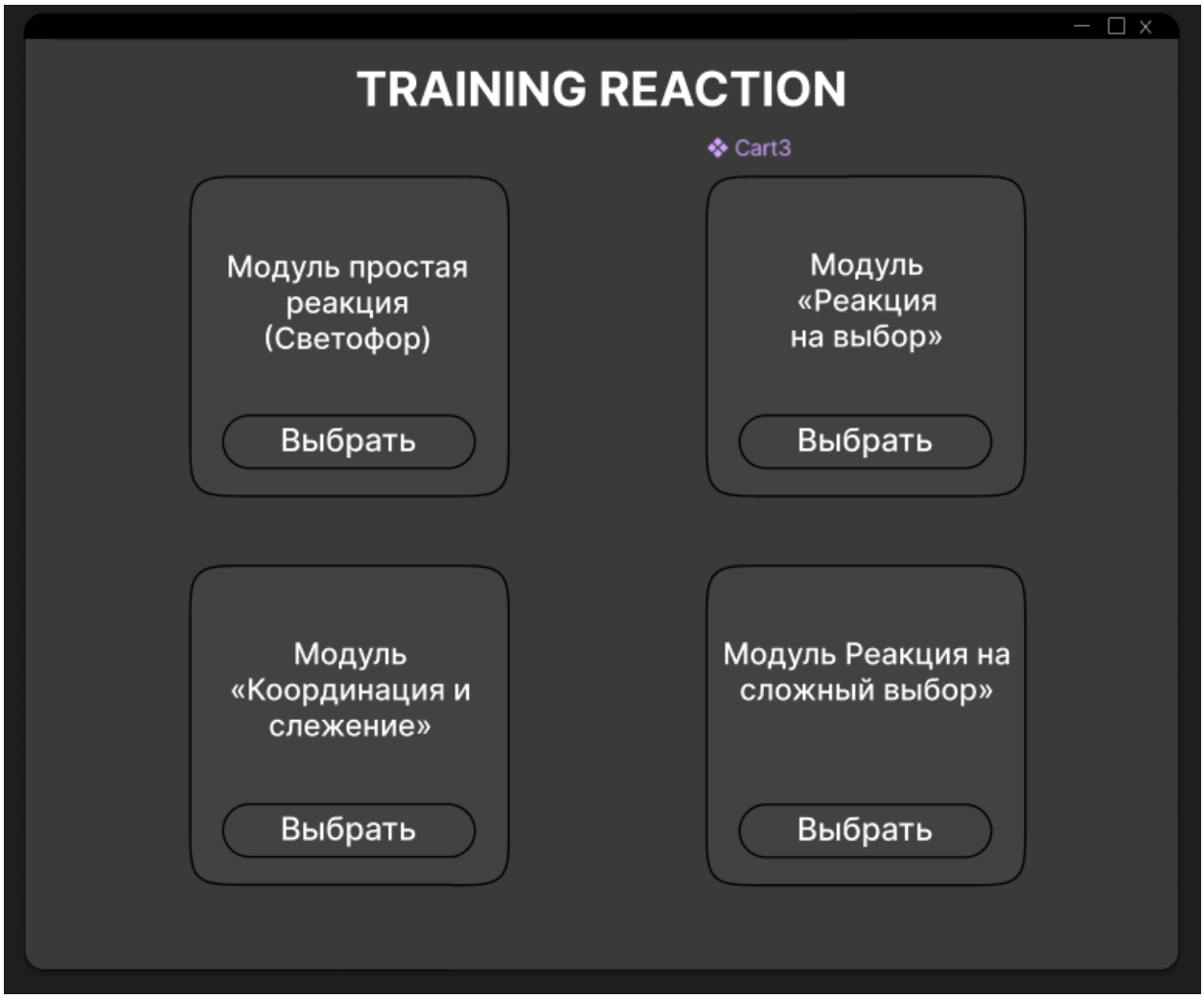

Рисунок 4.2.14 – Макет главного окна

Интерфейсное решение главного экрана реализовано в виде интерактивного каталога, состоящего из четырех функциональных карт. Карточки содержат наименование дисциплины и кнопку инициализации протокола. Такое разделение позволяет испытуемому быстро ориентироваться в структуре приложения, обеспечивая мгновенный переход к нужному алгоритму тестирования через единую навигационную панель.

Макет окна модуля «Простая реакция» приложения представлена на рисунке 4.2.15

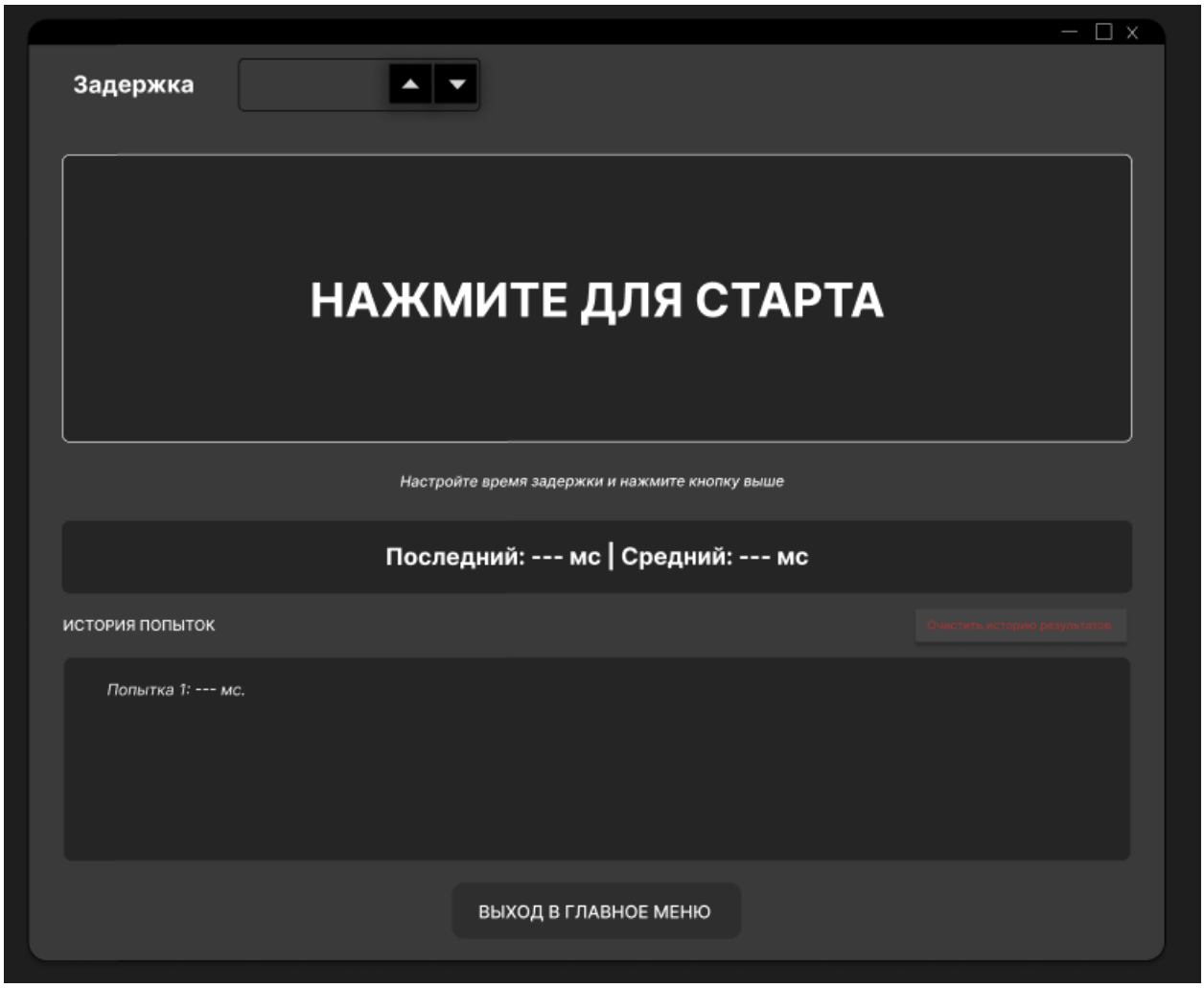

Рисунок 4.2.15 – Макет модуля «Простая реакция»

Интерфейс модуля «Простая сенсомоторная реакция» реализован в виде минималистичного рабочего пространства с центральным полем экспозиции. Макет включает в себя панель настроек временных задержек, информационное табло для вывода инструкций и крупную интерактивную зону, меняющую цветовой фон для подачи визуального стимула. Нижняя часть интерфейса отведена под блок текущей статистики, где отображаются результаты латентного периода последней попытки и средневзвешенное значение за всю сессию, а также вертикальный список истории для протоколирования динамики тренировки.

Макет окна модуля «Реакция на выбор» приложения представлена на рисунке 4.2.16

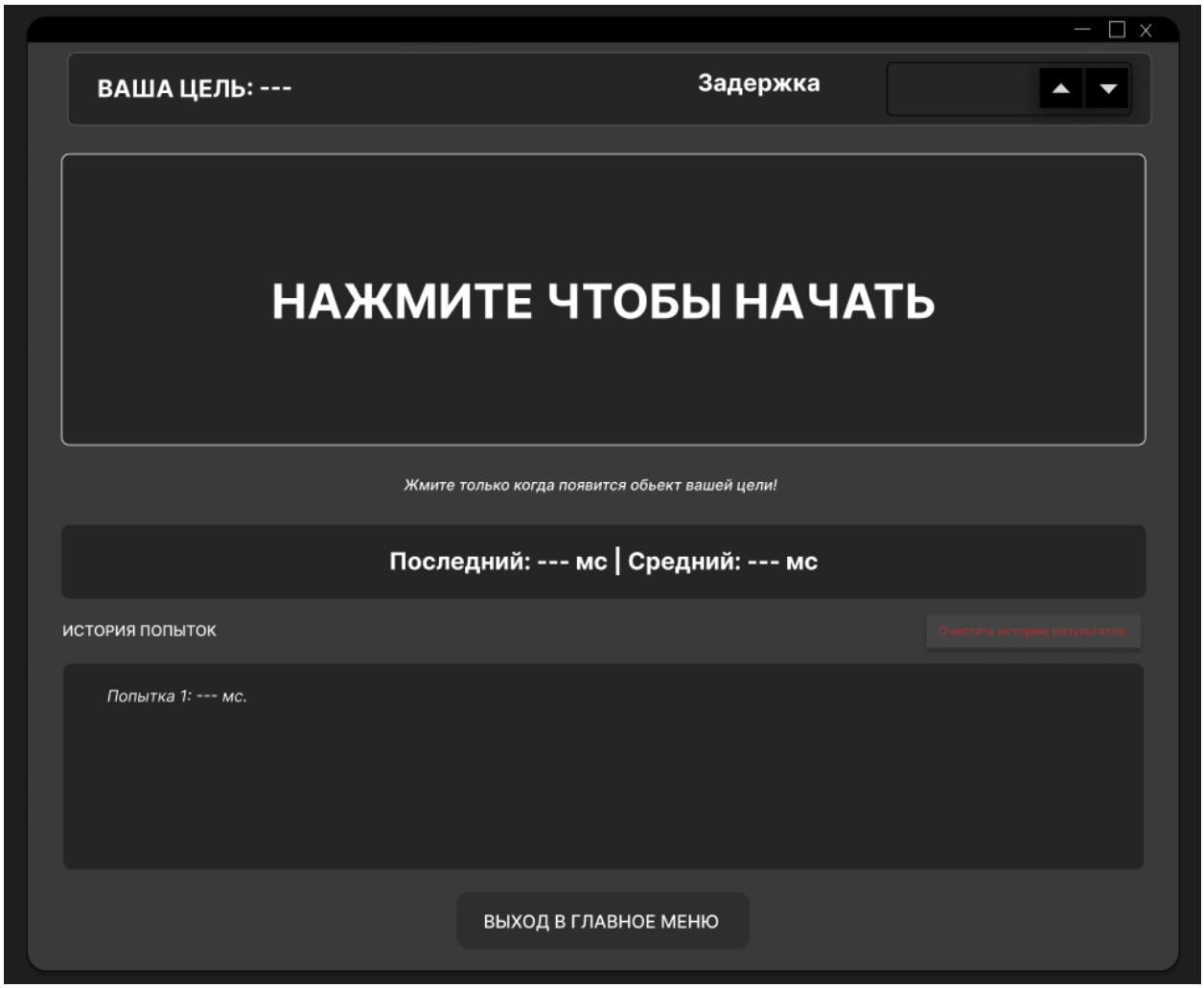

Рисунок 4.2.16 – Макет модуля «Реакция на выбор»

Интерфейс модуля «Дифференцировочная реакция» ориентирован на задачу селективного внимания и распознавания образов. Макет состоит из центральной области, в которой в случайном порядке генерируются различные геометрические стимулы, и боковой панели мониторинга результатов. Ключевым элементом является механизм визуальной дифференциации: испытуемый должен идентифицировать целевую фигуру среди отвлекающих объектов, при этом правильное и ошибочное действия сопровождаются мгновенной цветовой индикацией границ рабочего поля.

Макет окна модуля «Координация и слежение» приложения представлена на рисунке 4.2.17

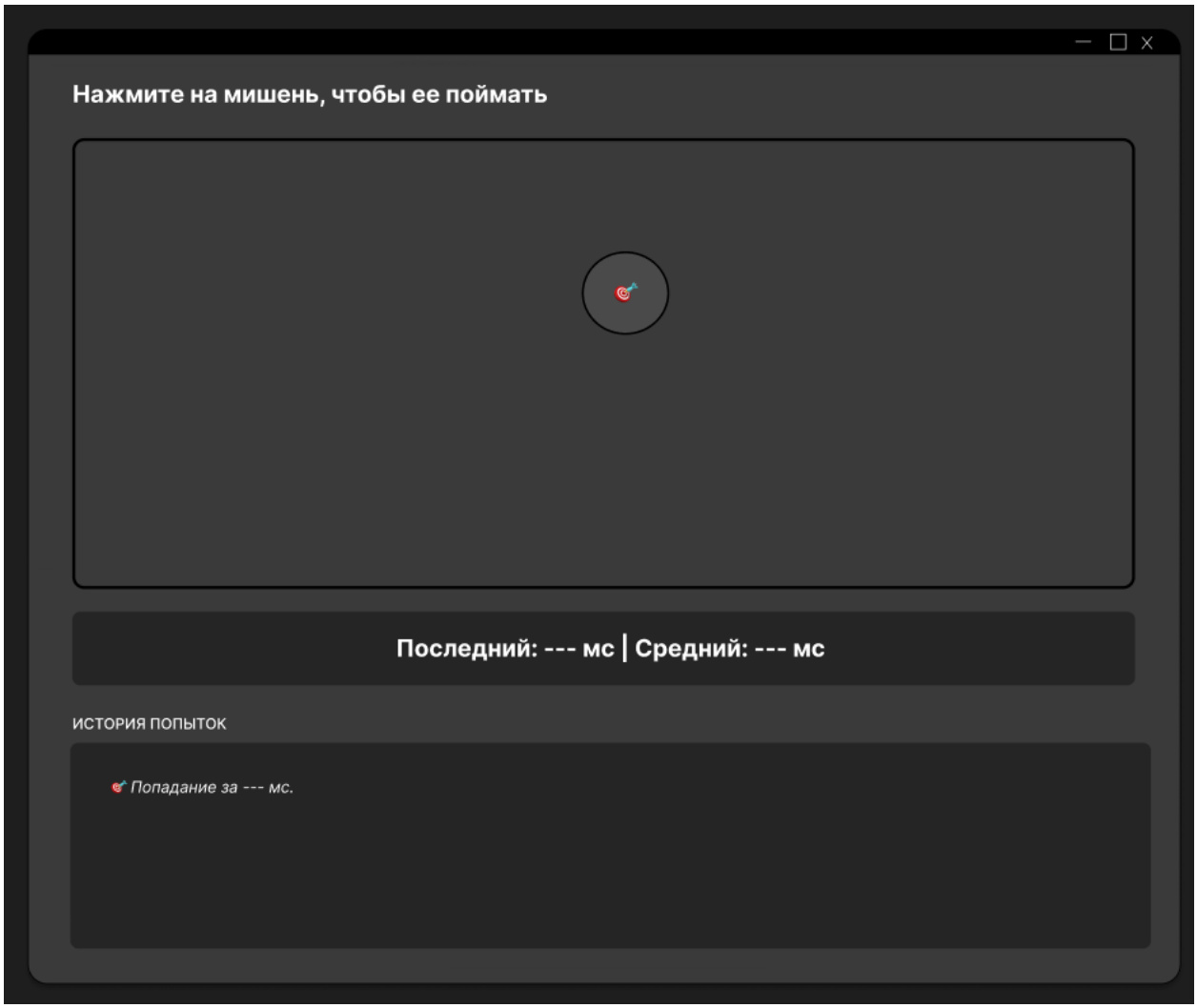

Рисунок 4.2.17 – Макет модуля «Координация и слежение»

Интерфейс модуля «Визуально-моторная координация» спроектирован для оценки точности и скорости взаимодействия с динамическими объектами. Рабочая область представляет собой расширенное поле, на котором в разных координатах дискретно возникают целевые маркеры. В отличие от статических тестов, данный макет минимизирует количество текстовой информации в центре экрана, чтобы не отвлекать внимание от перемещающегося стимула. Боковая панель управления позволяет задать диапазон задержек между появлениями.

Макет окна модуля «Двойной выбор» приложения представлена на рисунке 4.2.18

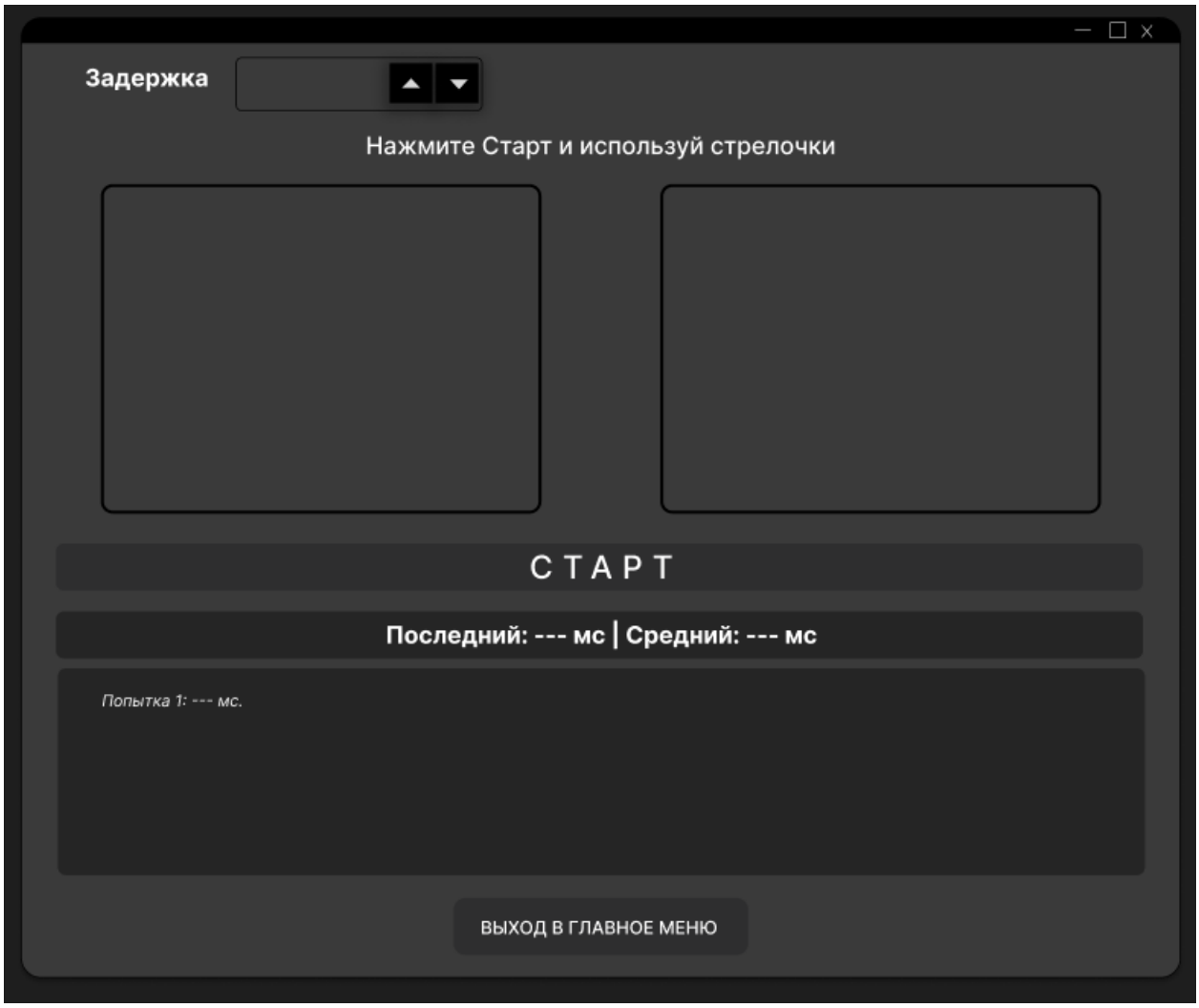

Рисунок 4.2.18 – Макет модуля «Двойной выбор»

Интерфейс модуля «Реакция на сложный выбор» спроектирован для раздельной оценки моторного отклика левой и правой руки при работе с клавиатурой. Макет включает две симметричные зоны визуальной индикации, которые активируются случайным образом, предъявляя буквенный стимул и вектор направления. Особенностью интерфейса является эргономика ввода: текстовые подсказки («A» и «L») интегрированы непосредственно в активные поля, чтобы минимизировать время на сопоставление визуального сигнала с физической клавишей.

Вывод

В данном разделе рассмотрены алгоритмы функционирования системы. Рассмотрены алгоритмы простой и дифференцировочной реакции, визуально-моторной координации, а также реакции на сложный выбор. На схемах алгоритмов представлены логические блоки, обеспечивающие прецизионный хронометраж и валидацию действий пользователя. Основной сложностью является обеспечение высокой точности регистрации отклика в условиях различных когнитивных нагрузок, в связи с чем в программном комплексе реализована система фильтрации аппаратных прерываний и фальстартов.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования программы, разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано на языке Python в среде PyCharm с использованием библиотеки PyQt6 для создания современного графического пользовательского интерфейса. Программное обеспечение имеет модульную архитектуру и состоит из нескольких основных классов, отвечающих за навигацию, отображение меню и логику отдельных тренажеров, а также отдельного модуля для работы с файловой системой.

Основным классом приложения является класс TrainingApp, наследуемый от QMainWindow. Создавая экземпляр этого класса, инициализируется главное окно программы. Класс содержит функции для маршрутизации переключения между окнами тренажеров с использованием виджета QStackedWidget.

В классе реализованы методы switch_screen(self, name) для перехода к конкретному модулю и go_to_menu(self) для возврата на главный экран. Сигнатура функции switch_screen принимает строковый параметр name, который является внутренним идентификатором выбранного тренажера.

Исходный код программного обеспечения в полном объеме представлен в приложении А. Исходный код функции switch_screen представлен на рисунке 4.3.1.

```
def switch_screen(self, name):
        """Переключение на экран игры по её кодовому имени"""
        if name == "traffic_light":
            self.stacked_widget.setCurrentIndex(1)
        elif name == "choice":
            self.stacked_widget.setCurrentIndex(2)
        elif name == "spatial":
            self.stacked_widget.setCurrentIndex(3)
        elif name == "dual":
            self.stacked_widget.setCurrentIndex(4)
```

Рисунок 4.3.1 – Исходный код функции switch_screen

Класс MenuWidget (наследуемый от QWidget) отвечает за отображение главного меню приложения. Внутри него используются экземпляры вспомогательного класса ModuleCard, который представляет собой визуальную карточку каждого тренажера с иконкой и кнопкой выбора. При нажатии на кнопку в классе ModuleCard генерируется сигнал clicked, передающий идентификатор модуля в главное окно для переключения интерфейса.

Класс TrafficLight реализует логику модуля «Простая реакция (Светофор)». Он содержит функции для запуска таймера ожидания start_waiting(), изменения визуального состояния экрана activate_light() и обработки реакции пользователя finish_round(). Для генерации случайной задержки перед появлением стимула используется встроенный класс QTimer. Фрагмент функции finish_round, рассчитывающий время реакции и вызывающий сохранение результата, представлен на рисунке 4.3.2.

```
def finish_round(self):
    reaction = (time.perf_counter() - self.start_perf_time) * 1000
    self.is_active = False
    self.results.append(reaction)
    save_result("Простая реакция", reaction)
    avg = sum(self.results) / len(self.results)
    res_text = f"{reaction:.1f} мс"
    self.screen.setText(res_text)
```

Рисунок 4.3.2 – Исходный код функции finish_round

Класс ChoiceReaction моделирует работу тренажера «Реакция на выбор». В классе реализован бесконечный цикл смены геометрических фигур с помощью таймера cycle_timer и функции next_step(). Функция check_reaction() выполняет проверку: совпадает ли текущая фигура на экране с целевой. Если фигуры совпадают, цикл останавливается и результат фиксируется, иначе вызывается функция trigger_error(), фиксирующая промах. Исходный код функции check_reaction представлен на рисунке 4.3.3.

```
def check_reaction(self):
        reaction = (time.perf_counter() - self.start_perf_time) * 1000
        if self.current_shape == self.target_shape:
            # Если успешно
            self.cycle_timer.stop()
            self.results.append(reaction)
            save_result("Реакция на выбор", reaction)

            avg = sum(self.results) / len(self.results)
            self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний: {avg:.1f})
            self.history_list.insertItem(0, f"✅ {self.target_shape}: {reaction:.1f} мс")
            self.screen.setText("🎯")
            self.hint_label.setText("Попали! Нажмите на поле для нового поиска")
            self.is_active = False
        else:
            # Ошибка
            self.trigger_error()
```

Рисунок 4.3.3 – Исходный код функции check_reaction

Класс DualReaction реализует тренажер «Реакция на сложный выбор». Особенностью данного класса является использование аппаратных прерываний клавиатуры путем переопределения встроенной функции keyPressEvent(event). В зависимости от того, в какой визуальной зоне (левой или правой) появился стимул, алгоритм проверяет соответствие нажатой клавиши (стрелка влево или вправо). Исходный код функции keyPressEvent(event) представлен на рисунке 4.3.4.

```
def keyPressEvent(self, event):
    if not self.is_running: return

    # Обработка фальстарта
    if not self.is_active:
        if event.key() in [Qt.Key.Key_Left, Qt.Key.Key_Right]:
            self.handle_error("ФАЛЬСТАРТ!")
        return

    reaction = (time.perf_counter() - self.start_perf_time) * 1000

    # Проверка правильности нажатия
    if event.key() == Qt.Key.Key_Left and self.target_side == "left":
        self.record_success(reaction, "Лево")
    elif event.key() == Qt.Key.Key_Right and self.target_side == "right":
        self.record_success(reaction, "Право")
    elif event.key() in [Qt.Key.Key_Left, Qt.Key.Key_Right]:
        self.handle_error("ОШИБКА СТОРОНЫ!")
```

Рисунок 4.3.4 – Исходный код функции keyPressEvent

Класс SpatialReaction представляет собой тренажер «Координация и слежение». Логика работы данного модуля базируется на функции spawn_target(), которая динамически вычисляет доступную площадь внутри выделенной игровой зоны и генерирует случайные координаты для перемещения элемента управления, выступающего в роли мишени. При успешном нажатии пользователем по мишени вызывается функция handle_target_click(), которая останавливает отсчет времени, сохраняет результат и немедленно вызывает подготовку следующей мишени для обеспечения непрерывности процесса тренировки. Исходный код функции handle_target_click представлен на рисунке 4.3.5.

```
def handle_target_click(self):
        if self.is_active and self.is_running:
            reaction = (time.perf_counter() - self.start_perf_time) * 1000
            self.is_active = False
            self.target.hide()

            self.results.append(reaction)
            save_result("Координация и слежение", reaction)

            avg = sum(self.results) / len(self.results)
            self.history_list.insertItem(0, f"🎯 Попадание: {reaction:.1f} мс")
            self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний: {avg:.1f})
            # Сразу зацикливаем, пока нажата кнопка СТОП
            self.prepare_next_target()
```

Рисунок 4.3.5 – Исходный код функции handle_target_click

Для обеспечения сохранности данных в программном обеспечении выделен отдельный функциональный файл storage.py. Данный скрипт не является классом, а представляет собой набор утилитарных функций для работы с файловой системой. Он содержит функции save_result(module_name, reaction_time), load_results(module_name), clear_module_history(module_name).

Использование этих функций позволяет записывать, считывать и удалять локальную статистику результатов пользователя в текстовый файл формата JSON. Структура хранения данных реализована в виде словаря, где ключами выступают названия модулей-тренажеров, а значениями – массивы с историей попыток (дата, время и результат в миллисекундах). Исходный код функции save_result представлен на рисунке 4.3.6.

```
def save_result(module_name, reaction_time):
    # Сохраняем новый результат в JSON-файл
    data = {}
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, 'r', encoding='utf-8') as f:
                data = json.load(f)
        except json.JSONDecodeError:
            pass

    if module_name not in data:
        data[module_name] = []

    # Добавляем новую запись
    data[module_name].append({
        "date": datetime.now().strftime("%d.%m %H:%M"),
        "reaction_ms": round(reaction_time, 1)
    })

    with open(FILE_NAME, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4, ensure_ascii=False)
```

Рисунок 4.3.6 – Исходный код функции save_result

Диаграмма классов представлена на рисунке 4.3.7.

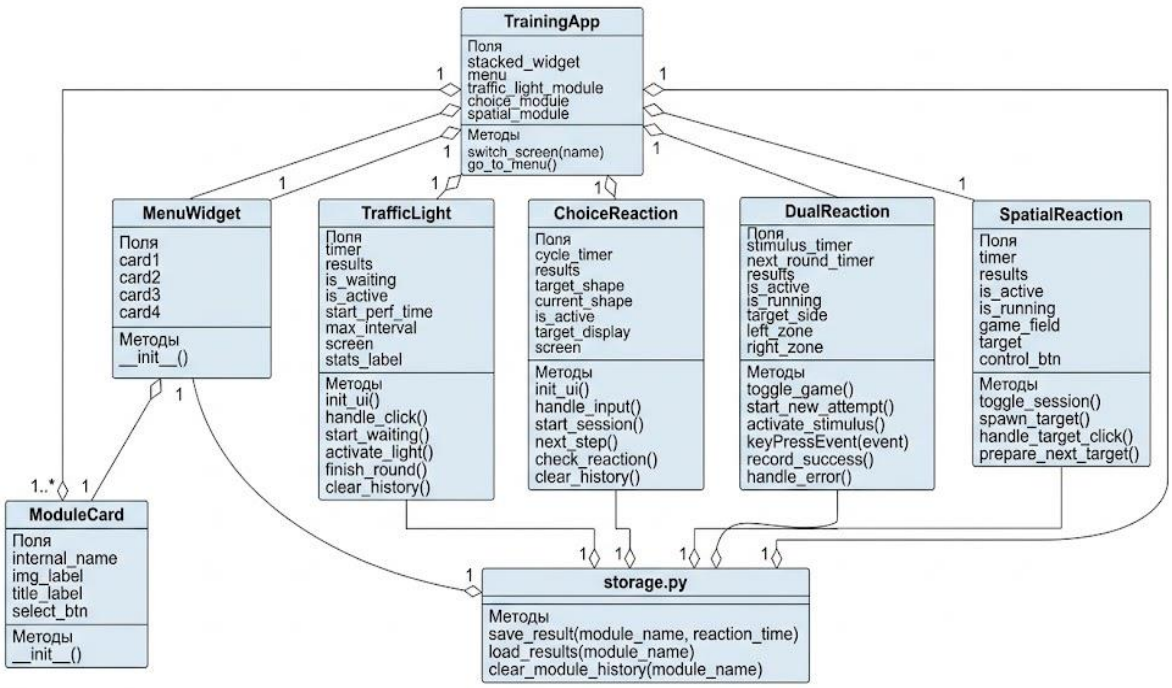

Рисунок 4.3.7 – Диаграмма классов

4.3.2 Разработка пользовательского интерфейса

Пользовательский интерфейс программного обеспечения разработан с использованием библиотеки PyQt6 для языка Python. Формирование экранных форм, расположение элементов управления и их стилизация производятся программно с применением компоновщиков (Layouts) и каскадных таблиц стилей (CSS) через метод setStyleSheet().

Главное окно приложения (меню выбора тренажеров) формируется в классе MenuWidget. Для организации пространства используются менеджеры компоновки QVBoxLayout (вертикальное расположение) и QGridLayout (выравнивание в виде сетки). Выбор конкретного тренажера осуществляется через интерактивные карточки (класс ModuleCard), базой для которых выступает контейнер QFrame. Внутри карточки расположены текстовые и графические метки QLabel, а также кнопка подтверждения выбора, реализованная через объект QPushButton. Главное окно приложения представлено на рисунке 4.3.8.

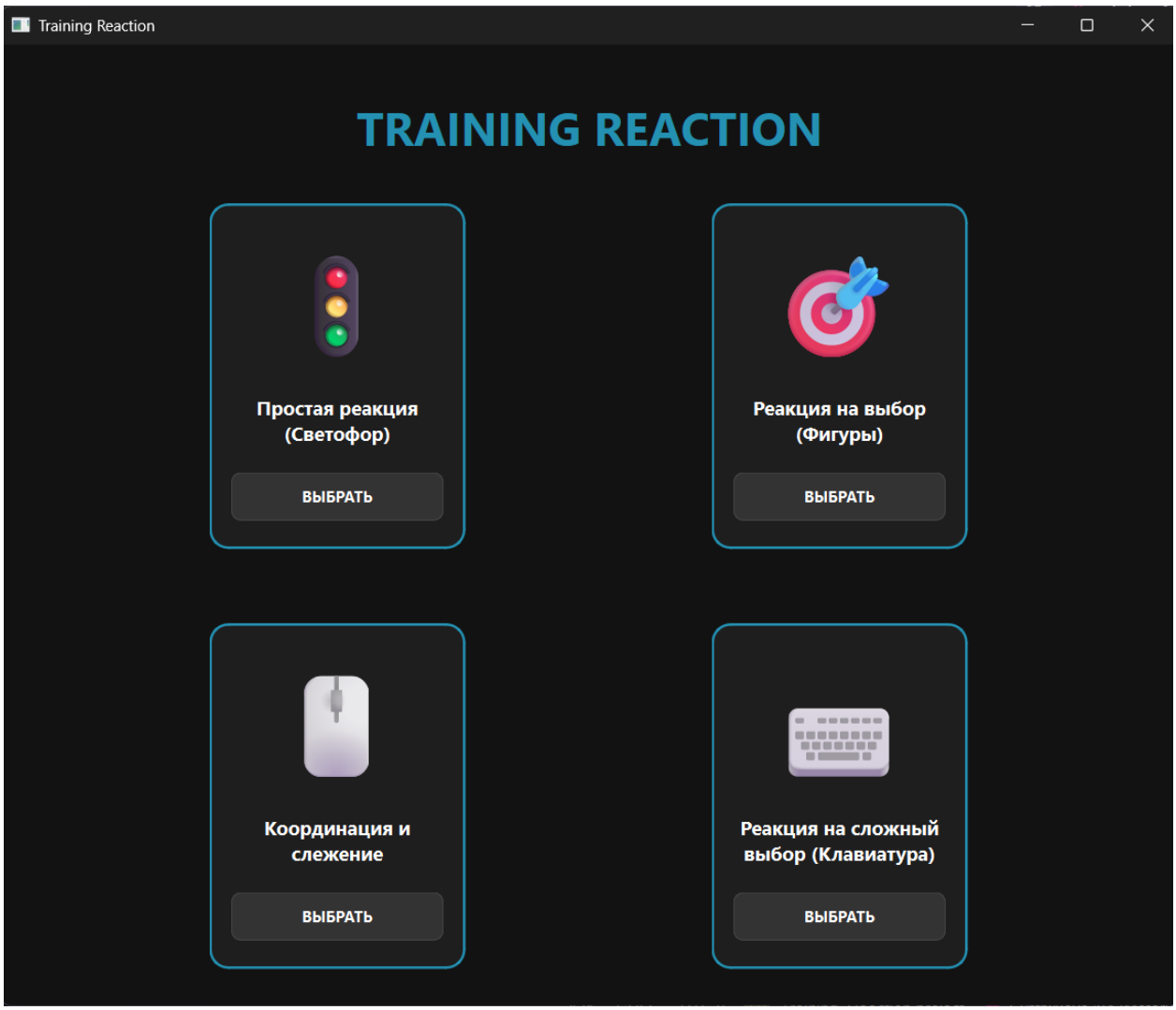

Рисунок 4.3.8 – Главное окно приложения

Экранные формы самих модулей-тренажеров имеют унифицированную визуальную структуру, состоящую из панели настроек, интерактивной игровой зоны, панели статистики и блока истории. Описание реализации визуальных объектов в тренажерах:

– QSpinBox используется на панели настроек для точного ввода числовых параметров (время задержки). Элемент ограничен минимальным и максимальным порогом, а также шагом изменения;

– QLabel применяется для вывода статической информации (заголовки), динамических подсказок по ходу тренировки и вывода текущей статистики (среднее время реакции);

– QListWidget – это визуальный объект списка, который служит для отображения истории попыток пользователя с возможностью вертикальной прокрутки;

– QPushButton – это основной элемент взаимодействия. Данный класс используется для создания кнопок возврата в меню, очистки истории, крупных интерактивных зон. Программа динамически меняет текст, цвет фона и границ кнопки в зависимости от текущего состояния (ожидание, стимул, фальстарт);

– QFrame используется для выделения логических блоков. В частности, в тренажере на сложный выбор с помощью него реализованы левая и правая визуальные зоны, которые подсвечиваются при появлении стимула.

В качестве примера реализации пользовательского интерфейса, на рисунке 4.3.9 приведены фрагменты исходного кода, демонстрирующие программную инициализацию и стилизацию (с помощью CSS-свойств) всех основных визуальных компонентов, используемых в модулях программного обеспечения.

```
# 1. Инициализация поля ввода числовых параметров (QSpinBox)
self.interval_spin = QSpinBox()
self.interval_spin.setRange(500, 10000)
self.interval_spin.setValue(4000)
self.interval_spin.setSingleStep(500)
self.interval_spin.setStyleSheet("""
    QSpinBox {background-color: #222; color: #2491b3; border: 1px solid #2491b3; border-
radius: 5px; padding: 5px; font-size: 14px;} """)
# 2. Инициализация текстовой метки (QLabel) для вывода информации
self.stats_label = QLabel("Последний: --- | Средний: ---")
self.stats_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.stats_label.setStyleSheet(""" font-size: 18px; color: white; font-weight: bold;
background: #222; padding: 10px; border-radius: 5px;""")
# 3. Инициализация интерактивной кнопки управления (QPushButton)
self.action_btn = QPushButton("СТАРТ ТРЕНИРОВКИ")
self.action_btn.setCursor(Qt.CursorShape.PointingHandCursor)
self.action_btn.setStyleSheet("""
    QPushButton {background-color: #2491b3; color: black; font-weight: bold; font-size:
18px; border-radius: 5px; padding: 10px;}
    QPushButton:hover {background-color: #4aa0ba;} """)
# 4. Инициализация визуальной зоны/контейнера (QFrame)
self.left_zone = QFrame()
self.left_zone.setFixedSize(380, 280)
self.left_zone.setStyleSheet(""" background-color: #1a1a1a; border: 2px solid #333;
border-radius: 15px;""")
# 5. Инициализация списка истории попыток (QListWidget)
self.history_list = QListWidget()
self.history_list.setFixedHeight(120)
self.history_list.setStyleSheet(""" background-color: #1a1a1a; color: #2491b3; border:
1px solid #333; border-radius: 10px; padding: 5px; """)
```

Рисунок 4.3.9. – Фрагменты кода визуальных объектов интерфейса

Экранные формы работы модулей программного обеспечения приведены на рисунках 4.3.10 – 4.3.13.

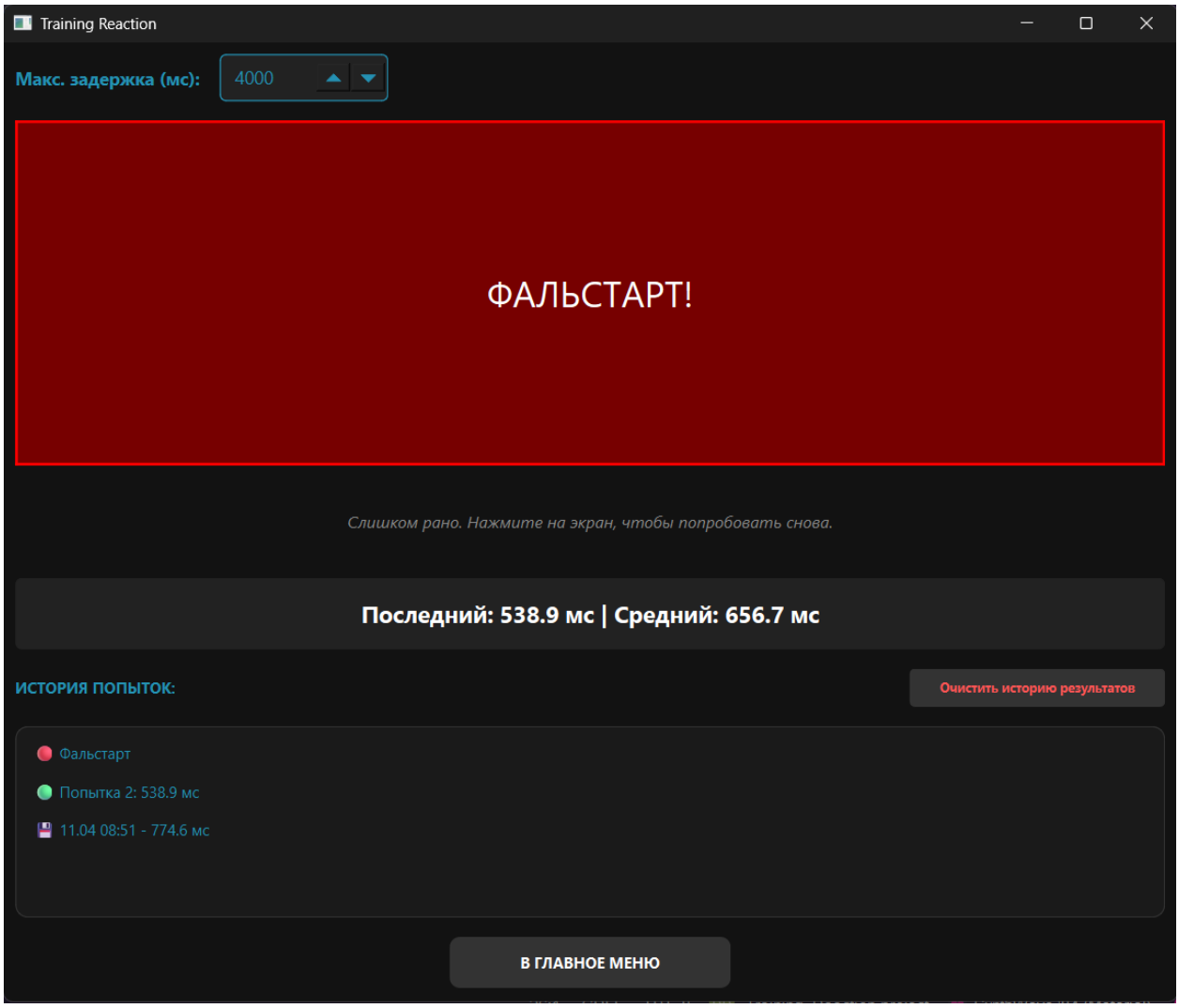

Рисунок 4.3.10 – Экранная форма модуля "Простая реакция"

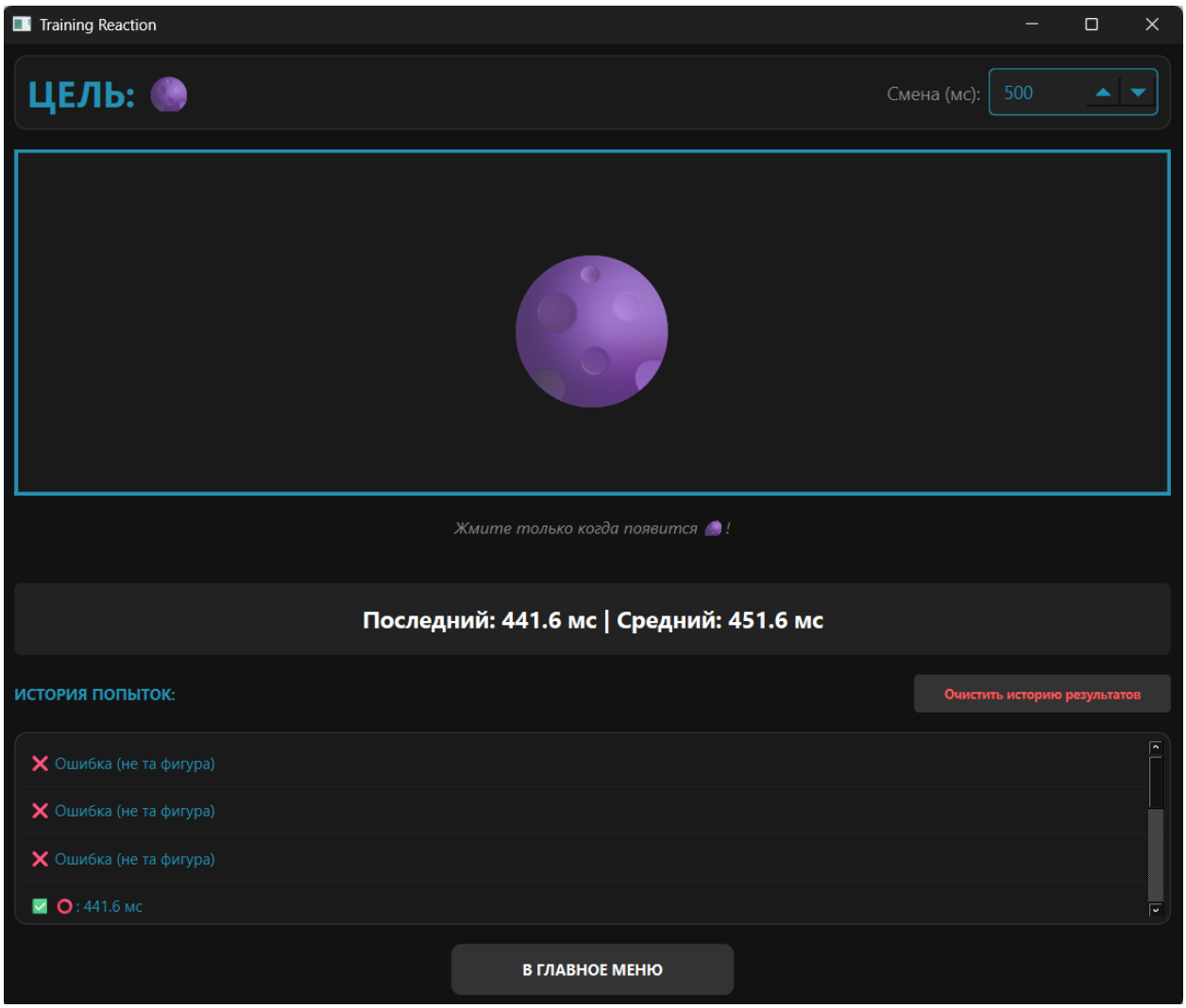

Рисунок 4.3.11 – Экранная форма модуля "Реакция на выбор"

Вывод: в ходе выполнения данного этапа учебной практики были успешно разработаны алгоритмы функционирования и исходный код программного обеспечения «Training Reaction». Приложение спроектировано на языке Python с использованием библиотеки PyQt6, что позволило создать современный, кроссплатформенный графический интерфейс.

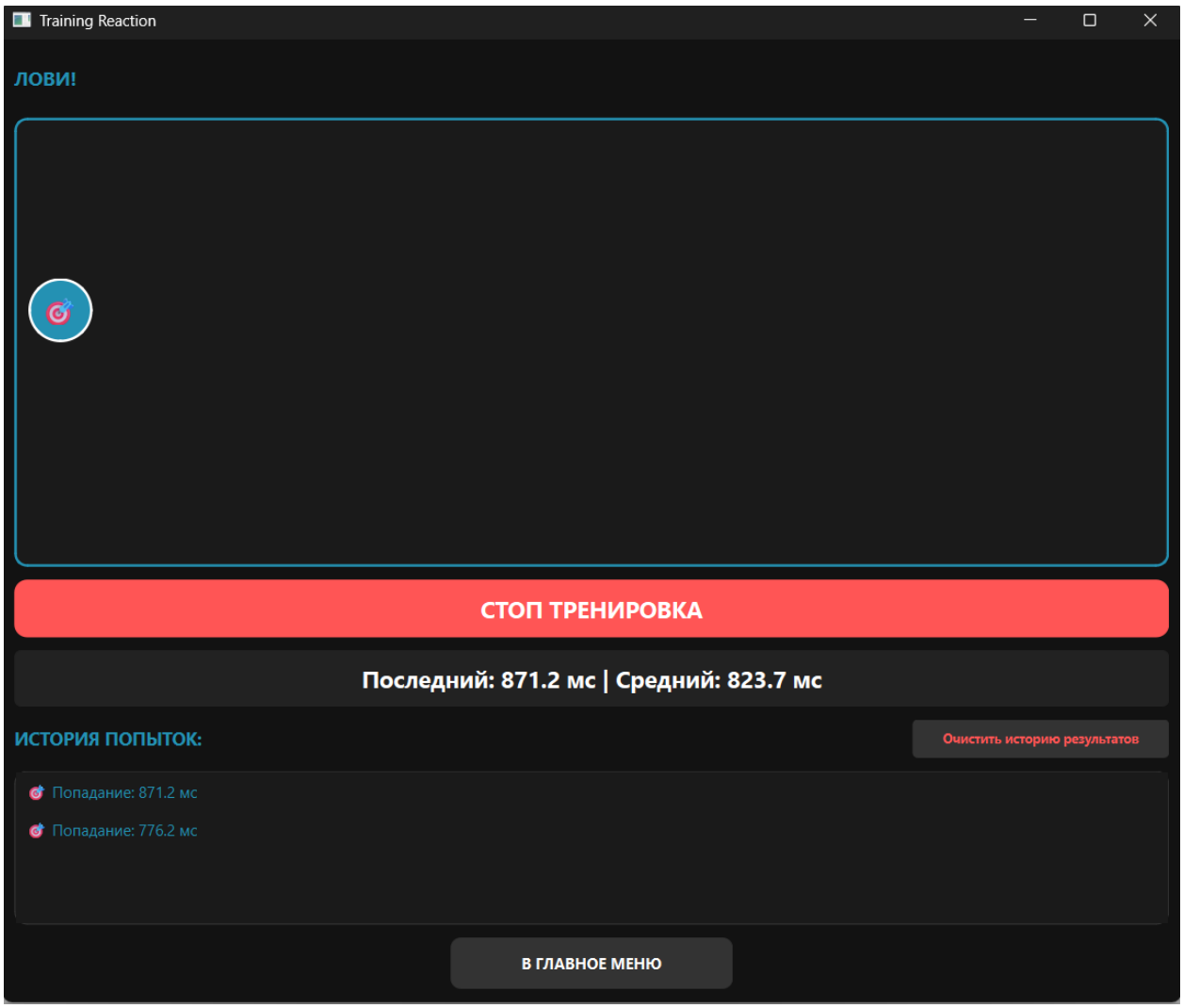

Рисунок 4.3.12 – Экранная форма модуля "Координация и слежение"

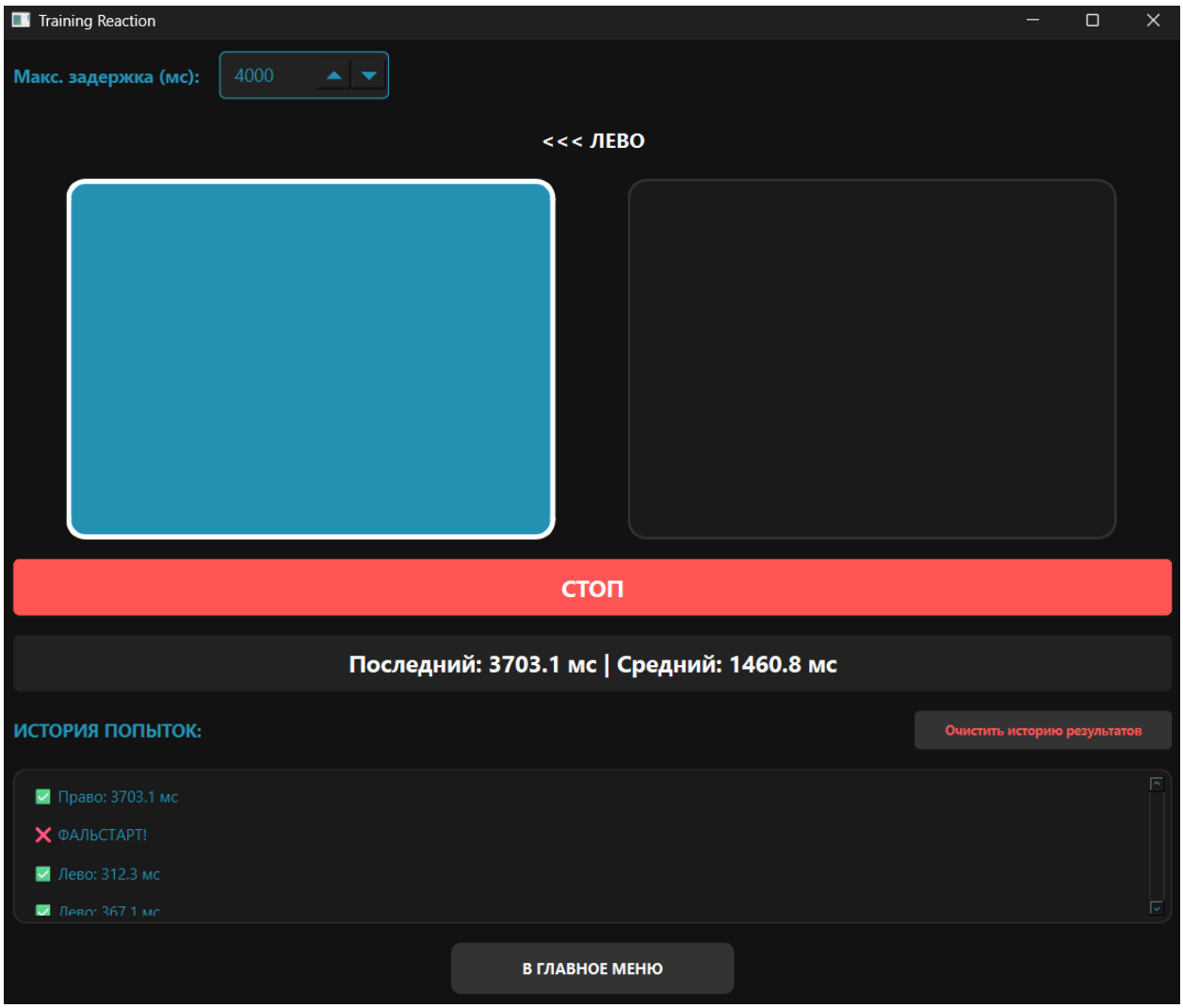

Рисунок 4.3.13 – Экранная форма модуля "Сложный выбор

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Дружинин, В. Н. «Экспериментальная психология: учебник для вузов» / Санкт-Петербург: Изд-во «Питер», 2001.

2. Солсо, Р. Л. «Когнитивная психология: учебник для вузов» / пер. с англ. – 6-е изд. – Санкт-Петербург: Изд-во «Питер», 2006.

3. Реакции человека и их характеристики / URL:https://azps.ru/articles/tezis/9reactions.html (дата обращения: 24.01.2026)

4. Ильин, Е.П. «Психомоторная организация человека» / Санкт-Петербург: Изд-во «Питер», 2003.

5. Журнал эпидемиологии / URL:https://academic.oup.com/aje/article/194/11/3208/7942379 (Дата обращения: 31.01.2026)

6. CogniFit Blog Тест оценки когнитивных способностей / URL: https://blog.cognifit.com/ru/cognifit-launched-an-innovative-cognitive-assessment-for-students-cab-k-12 (Дата обращения: 31.01.2026)

Приложение А

Исходный код

```
import sys
from PyQt6.QtWidgets import QApplication, QMainWindow, QStackedWidget

# Импортируем все наши 5 модулей (файлов)
from menu import MenuWidget
from Project.modules.traffic_light_module import TrafficLight
from Project.modules.choice_module import ChoiceReaction
from Project.modules.spatial_module import SpatialReaction
from Project.modules.dual_module import DualReaction

class TrainingApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Training Reaction")
        self.setMinimumSize(850, 750)
        self.setStyleSheet("background-color: #121212; color: white;")

        # Стек окон для переключения между меню и играми
        self.stacked_widget = QStackedWidget()
        self.setCentralWidget(self.stacked_widget)

        # 1. Создаем экземпляры окон
        self.menu = MenuWidget()
        self.traffic_light_module = TrafficLight()
        self.choice_module = ChoiceReaction()
        self.spatial_module = SpatialReaction()
        self.dual_module = DualReaction()

        # 2. Добавляем в стек (важен индекс!)
        self.stacked_widget.addWidget(self.menu)  # Индекс 0
        self.stacked_widget.addWidget(self.traffic_light_module)  # Индекс 1
        self.stacked_widget.addWidget(self.choice_module)  # Индекс 2
        self.stacked_widget.addWidget(self.spatial_module)  # Индекс 3
        self.stacked_widget.addWidget(self.dual_module)  # Индекс 4

        # 3. ПОДКЛЮЧЕНИЕ СИГНАЛОВ
        # Сигнал из меню (передает имя игры)
        self.menu.selected_module.connect(self.switch_screen)

        # Сигналы "Назад" из каждой игры
        self.traffic_light_module.back_to_menu.connect(self.go_to_menu)
        self.choice_module.back_to_menu.connect(self.go_to_menu)
        self.spatial_module.back_to_menu.connect(self.go_to_menu)
        self.dual_module.back_to_menu.connect(self.go_to_menu)

    def switch_screen(self, name):
        """Переключение на экран игры по её кодовому имени"""
        if name == "traffic_light":
            self.stacked_widget.setCurrentIndex(1)
        elif name == "choice":
            self.stacked_widget.setCurrentIndex(2)
        elif name == "spatial":
            self.stacked_widget.setCurrentIndex(3)
        elif name == "dual":
            self.stacked_widget.setCurrentIndex(4)

    def go_to_menu(self):
        """Возврат на главный экран"""
        self.stacked_widget.setCurrentIndex(0)
```

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = TrainingApp()
    window.show()
    sys.exit(app.exec())

from PyQt6.QtWidgets import QWidget, QGridLayout, QPushButton, QVBoxLayout, QLabel,
QFrame
from PyQt6.QtCore import Qt, pyqtSignal

class ModuleCard(QFrame):
    clicked = pyqtSignal(str)
    def __init__(self, title, internal_name, icon_emoji):
        super().__init__()
        self.internal_name = internal_name
        self.setFixedSize(200, 270)
        self.setStyleSheet("""
            QFrame {
                background-color: #1e1e1e;
                border: 2px solid #2491b3;
                border-radius: 15px;
            }
        """)
        layout = QVBoxLayout()
        layout.setContentsMargins(15, 20, 15, 20)
        layout.setSpacing(10)
        self.img_label = QLabel(icon_emoji)
        self.img_label.setStyleSheet("font-size: 80px; border: none; background:
transparent;")
        self.img_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.title_label = QLabel(title)
        self.title_label.setStyleSheet("color: white; font-size: 15px; font-weight: bold;
border: none;")
        self.title_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.title_label.setWordWrap(True)
        self.select_btn = QPushButton("ВЫБРАТЬ")
        self.select_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.select_btn.setStyleSheet("""
            QPushButton {
                background-color: #333; color: white; font-weight: bold;
                border-radius: 8px; padding: 10px; font-size: 12px; border: 1px solid
#444;
            }
            QPushButton:hover { background-color: #2491b3; color: black; border: 1px
solid #2491b3; }
        """)
        self.select_btn.clicked.connect(lambda: self.clicked.emit(self.internal_name))

        layout.addWidget(self.img_label)
        layout.addWidget(self.title_label)
        layout.addWidget(self.select_btn)
        self.setLayout(layout)

class MenuWidget(QWidget):
    selected_module = pyqtSignal(str)

    def __init__(self):
        super().__init__()
        layout = QVBoxLayout()
        layout.setContentsMargins(40, 40, 40, 40)

        header = QLabel("TRAINING REACTION")
        header.setStyleSheet("font-size: 36px; color: #2491b3; font-weight: bold; margin-
bottom: 30px;")
        header.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
        layout.addWidget(header)

        grid = QGridLayout()
        grid.setSpacing(70)

        # КАРТОЧКИ: Проверь internal_name у четвертой — теперь "sound"
        self.card1 = ModuleCard("Простая реакция (Светофор)", "traffic_light", "🚦")
        self.card2 = ModuleCard("Реакция на выбор (Фигуры)", "choice", "🎯")
        self.card3 = ModuleCard("Координация и слежение", "spatial", "🖱️")
        self.card4 = ModuleCard("Реакция на сложный выбор (Клавиатура)", "dual", "⌨️")

        self.card1.clicked.connect(self.selected_module.emit)
        self.card2.clicked.connect(self.selected_module.emit)
        self.card3.clicked.connect(self.selected_module.emit)
        self.card4.clicked.connect(self.selected_module.emit)

        grid.addWidget(self.card1, 0, 0)
        grid.addWidget(self.card2, 0, 1)
        grid.addWidget(self.card3, 1, 0)
        grid.addWidget(self.card4, 1, 1)

        layout.addLayout(grid)
        self.setLayout(layout)


import json
import os
from datetime import datetime

FILE_NAME = "reaction_stats.json"

def save_result(module_name, reaction_time):
    """Сохраняет новый результат в JSON-файл"""
    data = {}
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, 'r', encoding='utf-8') as f:
                data = json.load(f)
        except json.JSONDecodeError:
            pass

    if module_name not in data:
        data[module_name] = []

    # Добавляем новую запись
    data[module_name].append({
        "date": datetime.now().strftime("%d.%m %H:%M"),
        "reaction_ms": round(reaction_time, 1)
    })

    with open(FILE_NAME, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4, ensure_ascii=False)


def load_results(module_name):
    """Выгружает историю результатов из JSON-файла"""
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, 'r', encoding='utf-8') as f:
                data = json.load(f)
                return data.get(module_name, [])
        except json.JSONDecodeError:
            return []
    return []
```

```
def clear_module_history(module_name):
    """Безвозвратно удаляет историю конкретного модуля из файла"""
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, 'r', encoding='utf-8') as f:
                data = json.load(f)

            if module_name in data:
                data[module_name] = []  # Обнуляем список
                with open(FILE_NAME, 'w', encoding='utf-8') as f:
                    json.dump(data, f, indent=4, ensure_ascii=False)
        except json.JSONDecodeError:
            pass


import random
import time
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
QSpinBox, QListWidget
from PyQt6.QtCore import QTimer, Qt, pyqtSignal

from .storage import save_result, load_results, clear_module_history

class TrafficLight(QWidget):
    back_to_menu = pyqtSignal()

    def __init__(self):
        super().__init__()
        self.min_interval = 500
        self.max_interval = 15000

        self.is_waiting = False
        self.is_active = False
        self.start_perf_time = 0
        self.results = []

        self.timer = QTimer()
        self.timer.setSingleShot(True)
        self.timer.timeout.connect(self.activate_light)

        self.init_ui()
        self.load_history_from_file()

    def init_ui(self):
        # Основной вертикальный контейнер
        main_layout = QVBoxLayout()
        main_layout.setSpacing(15)  # Отступы между элементами

        # 1. Панель настроек (верх)
        settings_layout = QHBoxLayout()
        label_settings = QLabel("Макс. задержка (мс):")
        label_settings.setStyleSheet("color: #2491b3; font-weight: bold; font-size:
14px;")

        self.interval_spin = QSpinBox()
        self.interval_spin.setRange(500, 15000)
        self.interval_spin.setValue(self.max_interval)
        self.interval_spin.setSingleStep(500)
        self.interval_spin.setStyleSheet("""
            QSpinBox {
                background-color: #222;
                color: #2491b3;
                border: 1px solid #2491b3;
                border-radius: 5px;
```

```
                padding: 5px;
                font-size: 14px;
            }
        """)
        self.interval_spin.valueChanged.connect(self.update_settings)

        settings_layout.addWidget(label_settings)
        settings_layout.addWidget(self.interval_spin)
        settings_layout.addStretch()

        # 2. Игровое окно (центр)
        self.screen = QPushButton("НАЖМИТЕ ДЛЯ СТАРТА")
        self.screen.setFixedHeight(270)
        self.screen.setCursor(Qt.CursorShape.PointingHandCursor)
        self.screen.setStyleSheet("""
            QPushButton { background-color: #1a1a1a;  color: #2491b3; border: 2px solid
#2491b3;
                border-radius: 15px;
                font-size: 40px;
                font-weight: bold;
            }
        """)
        self.screen.clicked.connect(self.handle_click)

        # 3. Информационный лейбл (подсказки)
        self.hint_label = QLabel("Настройте время задержки и нажмите кнопку выше")
        self.hint_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.hint_label.setStyleSheet("color: #888; font-style: italic; font-size: 13px;
margin-bottom: 20px; margin-top: 20px;")

        # 4. Панель статистики
        self.stats_label = QLabel("Последний: --- | Средний: ---")
        self.stats_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.stats_label.setStyleSheet("""
            font-size: 18px; color: white; font-weight: bold;
            background: #222; padding: 16px; border-radius: 5px;
        """)

        # 5. Окно истории
        history_layout = QVBoxLayout()
        header_layout = QHBoxLayout() # (заголовок + кнопка)
        history_title = QLabel("ИСТОРИЯ ПОПЫТОК:")
        history_title.setStyleSheet("color: #2491b3; font-size: 12px; font-weight:
bold;")

        self.clear_btn = QPushButton("Очистить историю результатов")
        self.clear_btn.setFixedSize(200, 30)
        self.clear_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.clear_btn.setStyleSheet("""
            QPushButton {
                background-color: #333; color: #ff5555;
                border-radius: 4px; font-size: 10px; font-weight: bold;
            }
            QPushButton:hover { border: 1px solid #ff5555; }
        """)
        self.clear_btn.clicked.connect(self.clear_history)

        header_layout.addWidget(history_title)
        # header_layout.addStretch()
        header_layout.addWidget(self.clear_btn)

        self.history_list = QListWidget()
        self.history_list.setFixedHeight(150)
        self.history_list.setStyleSheet("""
            QListWidget { background-color: #1a1a1a; color: #2491b3; border: 1px solid
```

```
#333; border-radius: 10px; padding: 5px;}
        """)
        history_layout.addLayout(header_layout)
        history_layout.addWidget(self.history_list)

        # 6. Кнопка выхода
        self.back_btn = QPushButton("В ГЛАВНОЕ МЕНЮ")
        self.back_btn.setFixedSize(220, 40)
        self.back_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.back_btn.setStyleSheet("""
            QPushButton { background-color: #333; color: white; border-radius: 8px; font-
weight: bold; }
            QPushButton:hover { background-color: #444; border: 1px solid #2491b3; }
        """)
        self.back_btn.clicked.connect(self.back_to_menu.emit)

        # Сборка слоев
        main_layout.addLayout(settings_layout)
        main_layout.addWidget(self.screen)
        main_layout.addWidget(self.hint_label)
        main_layout.addWidget(self.stats_label)
        main_layout.addLayout(history_layout)
        main_layout.addWidget(self.back_btn, alignment=Qt.AlignmentFlag.AlignCenter)
        self.setLayout(main_layout)

    # Логика работы
    def update_settings(self):
        self.max_interval = self.interval_spin.value()

    def handle_click(self):
        if not self.is_waiting and not self.is_active:
            self.start_waiting()
        elif self.is_waiting:
            self.trigger_false_start()
        elif self.is_active:
            self.finish_round()

    def start_waiting(self):
        self.is_waiting = True
        self.is_active = False
        self.screen.setText("ЖДИТЕ...")
        self.screen.setStyleSheet("background-color: #222; color: #444; border: 2px solid
#444; font-size: 28px;")
        self.hint_label.setText("Приготовьтесь! Внимание на экран...")

        delay = random.randint(self.min_interval, self.max_interval)
        self.timer.start(delay)

    def activate_light(self):
        self.is_waiting = False
        self.is_active = True
        self.screen.setText("КЛИКАЙТЕ!")
        self.screen.setStyleSheet(
            "background-color: #2491b3; color: black; border: 3px solid white; font-size:
38px; font-weight: bold;")
        self.hint_label.setText("БЫСТРЕЕ!")
        self.start_perf_time = time.perf_counter()

    def trigger_false_start(self):
        self.timer.stop()
        self.is_waiting = False
        self.screen.setText("ФАЛЬСТАРТ!")
        self.screen.setStyleSheet("background-color: #770000; color: white; border: 2px
solid red; font-size: 28px;")
        self.hint_label.setText("Слишком рано. Нажмите на экран, чтобы попробовать
```

```
снова.")
        self.history_list.insertItem(0, "⚫ Фальстарт")

    def finish_round(self):
        reaction = (time.perf_counter() - self.start_perf_time) * 1000
        self.is_active = False
        self.results.append(reaction)
        save_result("Простая реакция", reaction)
        avg = sum(self.results) / len(self.results)

        res_text = f"{reaction:.1f} мс"
        self.screen.setText(res_text)
        self.screen.setStyleSheet(
            "background-color: #1a1a1a; color: #2491b3; border: 3px solid #2491b3; font-
size: 38px;")
        self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний: {avg:.1f} мс")
        self.hint_label.setText("Отличный результат! Нажмите еще раз, чтобы продолжить
сессию.")
        self.history_list.insertItem(0, f"□ Попытка {len(self.results)}: {res_text}")

    def clear_history(self):
        self.results = []
        self.history_list.clear()
        self.stats_label.setText("Последний: --- | Средний: ---")

        clear_module_history("Простая реакция")

        self.hint_label.setText("История очищена. Готовы к новому тесту?")
        self.screen.setText("НАЖМИТЕ ДЛЯ СТАРТА")
        self.screen.setStyleSheet("""
            QPushButton { background-color: #1a1a1a; color: #2491b3; border: 2px solid
#2491b3; border-radius: 15px; font-size: 26px; font-weight: bold; }
        """)

    def load_history_from_file(self):
        """Загружает данные из файла при запуске приложения"""
        saved_data = load_results("Простая реакция")

        for item in saved_data:
            reaction = item["reaction_ms"]
            date_str = item["date"]

            self.results.append(reaction)  # Добавляем в локальный массив для расчета
среднего
            self.history_list.insertItem(0, f"📄 {date_str} - {reaction:.1f} мс")

        # Если данные загрузились, обновляем табло статистики
        if self.results:
            avg = sum(self.results) / len(self.results)
            last = self.results[-1]
            self.stats_label.setText(f"Последний: {last:.1f} мс | Средний: {avg:.1f} мс")


import random
import time
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
QListWidget, QFrame
from PyQt6.QtCore import QTimer, Qt, pyqtSignal
from .storage import save_result, load_results, clear_module_history

class SpatialReaction(QWidget):
    back_to_menu = pyqtSignal()

    def __init__(self):
        super().__init__()
```

```
        self.results = []
        self.start_perf_time = 0
        self.is_active = False  # Видна ли мишень сейчас
        self.is_running = False  # Запущен ли процесс тренировки вообще

        self.timer = QTimer()
        self.timer.setSingleShot(True)
        self.timer.timeout.connect(self.spawn_target)

        self.init_ui()
        self.load_history_from_file()

    def init_ui(self):
        main_layout = QVBoxLayout()
        main_layout.setSpacing(10)

        # 1. Заголовок
        header_layout = QHBoxLayout()
        self.info_label = QLabel("Нажмите СТАРТ для начала тренировки")
        self.info_label.setStyleSheet("color: #2491b3; font-weight: bold; font-size:
15px;")
        header_layout.addWidget(self.info_label)
        main_layout.addLayout(header_layout)

        # 2. Игровая область (пустая, без кнопок внутри)
        self.game_field = QFrame()
        self.game_field.setFixedSize(900, 350)
        self.game_field.setStyleSheet("""
            QFrame {
                background-color: #1a1a1a;
                border: 2px solid #2491b3;
                border-radius: 10px;
            }
        """)
        # Мишень (скрыта по умолчанию)
        self.target = QPushButton("🎯", self.game_field)
        self.target.setFixedSize(50, 50)
        self.target.setCursor(Qt.CursorShape.CrossCursor)
        self.target.hide()
        self.target.setStyleSheet("""
            QPushButton {
                background-color: #2491b3;
                color: white;
                border-radius: 25px;
                font-size: 22px;
                border: 2px solid white;
            }
            QPushButton:hover { background-color: #2eb8e6; }
        """)
        self.target.clicked.connect(self.handle_target_click)
        main_layout.addWidget(self.game_field, alignment=Qt.AlignmentFlag.AlignCenter)

        # 3. Единая кнопка управления СТАРТ / СТОП
        self.control_btn = QPushButton("СТАРТ ТРЕНИРОВКИ")
        self.control_btn.setFixedHeight(45)
        self.control_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        # Стиль по умолчанию (СТАРТ)
        self.control_btn.setStyleSheet("""
            QPushButton {
                background-color: #2491b3;
                color: #1a1a1a;
                border-radius: 10px;
                font-size: 18px;
                font-weight: bold;
            }
```

```
            QPushButton:hover {
                background-color: #4aa0ba;
            }
        """)
        self.control_btn.clicked.connect(self.toggle_session)
        main_layout.addWidget(self.control_btn)

        # 4. Панель статистики
        self.stats_label = QLabel("Последний: --- | Средний: ---")
        self.stats_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.stats_label.setStyleSheet("font-size: 18px; color: white; font-weight: bold;
background: #222; padding: 10px; border-radius: 5px;")
        main_layout.addWidget(self.stats_label)

        # 5. Кнопка очистки (Справа)
        history_layout = QVBoxLayout()
        history_header = QHBoxLayout()

        history_title = QLabel("ИСТОРИЯ ПОПЫТОК:")
        history_title.setStyleSheet("color: #2491b3; font-weight:bold; font-size: 14px")

        self.clear_btn = QPushButton("Очистить историю результатов")
        self.clear_btn.setFixedSize(200, 30)
        self.clear_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.clear_btn.setStyleSheet("""
            QPushButton {
                background-color: #333; color: #ff5555;
                border-radius: 4px; font-size: 10px; font-weight: bold;
            }
            QPushButton:hover { border: 1px solid #ff5555; }
        """)
        self.clear_btn.clicked.connect(self.clear_history)

        history_header.addWidget(history_title)
        history_header.addStretch()
        history_header.addWidget(self.clear_btn)

        # 6. Список истории
        self.history_list = QListWidget()
        self.history_list.setFixedHeight(120)
        self.history_list.setStyleSheet(
            "background-color: #1a1a1a; color: #2491b3; border: 1px solid #333; border-
radius: 10px;")

        history_layout.addLayout(history_header)
        history_layout.addWidget(self.history_list)
        main_layout.addLayout(history_layout)

        # 7. Навигация
        self.back_btn = QPushButton("В ГЛАВНОЕ МЕНЮ")
        self.back_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.back_btn.setFixedSize(220, 40)
        self.back_btn.setStyleSheet("""
            QPushButton {background-color: #333; color: white; border-radius: 8px; font-
weight: bold; }
            QPushButton:hover { background-color: #444; border: 1px solid #2491b3; }
        """)
        self.back_btn.clicked.connect(self.back_to_menu.emit)
        main_layout.addWidget(self.back_btn, alignment=Qt.AlignmentFlag.AlignCenter)

        self.setLayout(main_layout)

    def toggle_session(self):
        # Переключение между режимами Старт и Стоп
        if not self.is_running:
```

```
            self.is_running = True # Запуск
            self.control_btn.setText("СТОП ТРЕНИРОВКА")
            self.control_btn.setStyleSheet("""
                QPushButton { background-color: #ff5555; color: white; border-radius:
10px; font-size: 18px; font-weight: bold; }
            """)
            self.info_label.setText("Приготовьтесь...")
            self.prepare_next_target()
        else:
            # Остановка (стиль СТАРТ из dual_module)
            self.is_running = False
            self.control_btn.setText("СТАРТ ТРЕНИРОВКИ")
            self.control_btn.setStyleSheet("""
                QPushButton { background-color: #2491b3; color: #1a1a1a; border-radius:
10px; font-size: 18px; font-weight: bold; }
                QPushButton:hover { background-color: #4aa0ba; }
            """)
            self.timer.stop()
            self.target.hide()
            self.is_active = False
            self.info_label.setText("Тренировка приостановлена")

    def prepare_next_target(self):
        if self.is_running:
            delay = random.randint(1000, 2500)
            self.timer.start(delay)

    def spawn_target(self):
        if not self.is_running:
            return

        max_x = self.game_field.width() - self.target.width() - 10
        max_y = self.game_field.height() - self.target.height() - 10

        new_x = random.randint(10, max_x)
        new_y = random.randint(10, max_y)

        self.target.move(new_x, new_y)
        self.target.show()
        self.info_label.setText("ЛОВИ!")
        self.start_perf_time = time.perf_counter()
        self.is_active = True

    def handle_target_click(self):
        if self.is_active and self.is_running:
            reaction = (time.perf_counter() - self.start_perf_time) * 1000
            self.is_active = False
            self.target.hide()

            self.results.append(reaction)
            save_result("Координация и слежение", reaction)

            avg = sum(self.results) / len(self.results)
            self.history_list.insertItem(0, f"🎯 Попадание: {reaction:.1f} мс")
            self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний: {avg:.1f}
мс")

            # Сразу зацикливаем, пока нажата кнопка СТОП
            self.prepare_next_target()

    def clear_history(self):
        self.results = []
        self.history_list.clear()
        self.stats_label.setText("Последний: --- | Средний: ---")
        self.info_label.setText("История очищена")
```

```
        clear_module_history("Координация и слежение")

    def load_history_from_file(self):
        """Загружает данные из файла при запуске приложения"""
        saved_data = load_results("Координация и слежение")

        for item in saved_data:
            reaction = item["reaction_ms"]
            date_str = item["date"]

            self.results.append(reaction)
            self.history_list.insertItem(0, f"💾 {date_str} - {reaction:.1f} мс")

        if self.results:
            avg = sum(self.results) / len(self.results)
            last = self.results[-1]
            self.stats_label.setText(f"Последний: {last:.1f} мс | Средний: {avg:.1f} мс")


import random
import time
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
QSpinBox, QListWidget, QFrame
from PyQt6.QtCore import QTimer, Qt, pyqtSignal
from .storage import save_result, load_results, clear_module_history

class ChoiceReaction(QWidget):
    back_to_menu = pyqtSignal()

    def __init__(self):
        super().__init__()
        self.shapes = ["⭘", "◉", "🔘", "🔘", "●", "○"]
        self.target_shape = ""
        self.current_shape = ""
        self.results = []
        self.start_perf_time = 0
        self.is_active = False

        self.cycle_timer = QTimer()
        self.cycle_timer.timeout.connect(self.next_step)

        self.init_ui()
        self.load_history_from_file()

    def init_ui(self):
        main_layout = QVBoxLayout()
        main_layout.setSpacing(15)

        # 1. Панель настроек (верх)
        top_frame = QFrame()
        top_frame.setStyleSheet("background: #1a1a1a; border-radius: 10px; border: 1px
solid #333;")
        top_layout = QHBoxLayout(top_frame)

        self.target_display = QLabel("ЦЕЛЬ: ---")
        self.target_display.setStyleSheet("color: #2491b3; font-weight: bold; font-
size: 28px; border: none;")

        speed_label = QLabel("Смена (мс):")
        speed_label.setStyleSheet("color: #888; border: none; font-size: 14px;")
        self.speed_spin = QSpinBox()
        self.speed_spin.setRange(500, 15000)
        self.speed_spin.setSingleStep(500)  # По умолчанию чуть быстрее для сложности
        self.speed_spin.setStyleSheet("background: #222; color: #2491b3; border: 1px
solid #2491b3; border-radius: 5px; padding: 5px; font-size: 14px;")
```

```
        top_layout.addWidget(self.target_display)
        top_layout.addStretch()
        top_layout.addWidget(speed_label)
        top_layout.addWidget(self.speed_spin)

        # 2. Игровое окно (центр)
        self.screen = QPushButton("НАЖМИТЕ ДЛЯ СТАРТА")
        self.screen.setFixedHeight(270)
        self.screen.setCursor(Qt.CursorShape.PointingHandCursor)
        self.screen.setStyleSheet("""
            QPushButton { background-color: #1a1a1a; color: #2491b3; border: 2px solid
#2491b3; border-radius: 15px; font-size: 40px; font-weight: bold;}
        """)
        self.screen.clicked.connect(self.handle_input)

        # 3. Информационный лейбл (подсказки)
        self.hint_label = QLabel("Нажмите СТАРТ, чтобы начать поиск")
        self.hint_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.hint_label.setStyleSheet("color: #888; font-size: 13px; font-style:
italic; margin-bottom: 20px; margin-top: 20px; ")

        # 4. Панель статистики
        self.stats_label = QLabel("Последний: --- | Средний: ---")
        self.stats_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.stats_label.setStyleSheet("""
            font-size: 18px; color: white; font-weight: bold;
            background: #222; padding: 16px; border-radius: 5px;
        """)

        # 5. Окно истории
        history_layout = QVBoxLayout()
        history_header = QHBoxLayout()
        hist_title = QLabel("ИСТОРИЯ ПОПЫТОК:")
        hist_title.setStyleSheet("color: #2491b3; font-size: 12px; font-weight:
bold;")

        self.clear_btn = QPushButton("Очистить историю результатов")
        self.clear_btn.setFixedSize(200, 30)
        self.clear_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.clear_btn.setStyleSheet("""
            QPushButton {
                background-color: #333; color: #ff5555;
                border-radius: 4px; font-size: 10px; font-weight: bold;
            }
            QPushButton:hover { border: 1px solid #ff5555; }
        """)
        self.clear_btn.clicked.connect(self.clear_history)

        history_header.addWidget(hist_title)
        history_header.addStretch()
        history_header.addWidget(self.clear_btn)

        self.history_list = QListWidget()
        self.history_list.setFixedHeight(150)
        self.history_list.setStyleSheet("""
            QListWidget { background-color: #1a1a1a; color: #2491b3; border: 1px solid
#333; border-radius: 10px; padding: 5px;}
            QListWidget::item { border-bottom: 1px solid #222; padding: 3px; }
        """)

        history_layout.addLayout(history_header)
        history_layout.addWidget(self.history_list)

        # 6. Кнопка выхода
```

```
        self.back_btn = QPushButton("В ГЛАВНОЕ МЕНЮ")
        self.back_btn.setFixedSize(220, 40)
        self.back_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.back_btn.setStyleSheet("""
                    QPushButton { background-color: #333; color: white; border-radius:
8px; font-weight: bold; }
                    QPushButton:hover { background-color: #444; border: 1px solid
#2491b3; }
                """)
        self.back_btn.clicked.connect(self.back_to_menu_action)

        # Сборка слоев
        main_layout.addWidget(top_frame)
        main_layout.addWidget(self.screen)
        main_layout.addWidget(self.hint_label)
        main_layout.addWidget(self.stats_label)
        main_layout.addLayout(history_layout)
        main_layout.addWidget(self.back_btn, alignment=Qt.AlignmentFlag.AlignCenter)
        self.setLayout(main_layout)

    def handle_input(self):
        if not self.cycle_timer.isActive():
            self.start_session()
        elif self.is_active:
            self.check_reaction()

    def start_session(self):
        self.target_shape = random.choice(self.shapes)
        self.target_display.setText(f"ЦЕЛЬ: {self.target_shape}")
        self.screen.setText("...")
        self.screen.setStyleSheet("background-color: #111; border: 3px solid #444;
color: #444; font-size: 120px;")
        self.hint_label.setText(f"Жмите только когда появится {self.target_shape}!")
        self.is_active = False
        self.cycle_timer.start(self.speed_spin.value())

    def next_step(self):
        # 25% шанс появления цели для усложнения ожидания
        if random.random() < 0.25:
            self.current_shape = self.target_shape
        else:
            self.current_shape = random.choice([s for s in self.shapes if s !=
self.target_shape])

        self.screen.setText(self.current_shape)
        self.screen.setStyleSheet(
            "background-color: #1a1a1a; border: 3px solid #2491b3; color: white; font-
size: 120px;")
        self.start_perf_time = time.perf_counter()
        self.is_active = True

    def check_reaction(self):
        reaction = (time.perf_counter() - self.start_perf_time) * 1000
        if self.current_shape == self.target_shape:
            # УСПЕХ
            self.cycle_timer.stop()
            self.results.append(reaction)
            save_result("Реакция на выбор", reaction)

            avg = sum(self.results) / len(self.results)
            self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний:
{avg:.1f} мс")
            self.history_list.insertItem(0, f"✅ {self.target_shape}: {reaction:.1f}
мс")
            self.screen.setText("🎯")
```

```
                self.hint_label.setText("Попали! Нажмите на поле для нового поиска.")
                self.is_active = False
            else:
                # ОШИБКА
                self.trigger_error()

        def trigger_error(self):
            self.cycle_timer.stop()
            self.screen.setText("✕")
            self.screen.setStyleSheet("background-color: #500; color: white; border: 3px
   solid red; font-size: 120px;")
            self.history_list.insertItem(0, "✕ Ошибка (не та фигура)")
            self.hint_label.setText("Это была не цель! Попробуйте снова.")
            self.is_active = False

        def clear_history(self):
            self.results = []
            self.history_list.clear()
            self.stats_label.setText("Последний: --- | Средний: ---")
            clear_module_history("Реакция на выбор")

        def back_to_menu_action(self):
            self.cycle_timer.stop()
            self.back_to_menu.emit()

        def load_history_from_file(self):
            """Загружает данные из файла при запуске приложения"""
            saved_data = load_results("Реакция на выбор")

            for item in saved_data:
                reaction = item["reaction_ms"]
                date_str = item["date"]
                self.results.append(reaction)
                self.history_list.insertItem(0, f"💾 {date_str} - {reaction:.1f} мс")
            if self.results:
                avg = sum(self.results) / len(self.results)
                last = self.results[-1]
                self.stats_label.setText(f"Последний: {last:.1f} мс | Средний: {avg:.1f}
   мс")


import random
import time
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
QFrame, QListWidget, QSpinBox
from PyQt6.QtCore import Qt, pyqtSignal, QTimer

from .storage import save_result, load_results, clear_module_history

class DualReaction(QWidget):
    back_to_menu = pyqtSignal()

    def __init__(self):
        super().__init__()
        self.results = []
        self.is_active = False  # Горит ли сейчас вспышка
        self.is_running = False  # Запущен ли бесконечный цикл игры
        self.start_perf_time = 0
        self.target_side = ""

        # Таймер для задержки появления вспышки
        self.stimulus_timer = QTimer()
        self.stimulus_timer.setSingleShot(True)
        self.stimulus_timer.timeout.connect(self.activate_stimulus)
```

```
        # Таймер для паузы МЕЖДУ раундами (чтобы результаты не слипались)
        self.next_round_timer = QTimer()
        self.next_round_timer.setSingleShot(True)
        self.next_round_timer.timeout.connect(self.start_new_attempt)

        self.init_ui()
        self.setFocusPolicy(Qt.FocusPolicy.StrongFocus)
        self.load_history_from_file()

    def init_ui(self):
        main_layout = QVBoxLayout()
        main_layout.setSpacing(15)

        # 1. ПАНЕЛЬ НАСТРОЕК
        settings_layout = QHBoxLayout()
        label_delay = QLabel("Макс. задержка (мс):")
        label_delay.setStyleSheet("color: #2491b3; font-weight: bold; font-size: 14px;")

        self.interval_spin = QSpinBox()
        self.interval_spin.setRange(1000, 10000)
        self.interval_spin.setValue(4000)
        self.interval_spin.setSingleStep(500)
        self.interval_spin.setStyleSheet("""
            QSpinBox {
                background-color: #222; color: #2491b3;
                border: 1px solid #2491b3; border-radius: 5px;
                padding: 5px; font-size: 14px;
            }
        """)

        settings_layout.addWidget(label_delay)
        settings_layout.addWidget(self.interval_spin)
        settings_layout.addStretch()

        # 2. ИНСТРУКЦИЯ
        self.info_label = QLabel("Нажми СТАРТ и используй стрелочки ← (влево) и →
(вправо)")
        self.info_label.setStyleSheet("color: white; font-size: 16px; font-weight:
bold;")
        self.info_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

        # 3. ИГРОВЫЕ ЗОНЫ
        field_layout = QHBoxLayout()
        self.left_zone = QFrame()
        self.left_zone.setFixedSize(380, 280)
        self.left_zone.setStyleSheet("background-color: #1a1a1a; border: 2px solid #333;
border-radius: 15px;")

        self.right_zone = QFrame()
        self.right_zone.setFixedSize(380, 280)
        self.right_zone.setStyleSheet("background-color: #1a1a1a; border: 2px solid #333;
border-radius: 15px;")

        field_layout.addWidget(self.left_zone)
        field_layout.addWidget(self.right_zone)

        # 4. КНОПКА УПРАВЛЕНИЯ (СТАРТ/СТОП)
        self.action_btn = QPushButton("СТАРТ ТРЕНИРОВКИ")
        self.action_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.action_btn.setStyleSheet("""
                    QPushButton {
                        background-color: #2491b3; color: black;
                        font-weight: bold; font-size: 18px; border-radius: 5px; padding:
10px;
                    }
```

```
                    QPushButton:hover { background-color: #4aa0ba; }
                """)
        self.action_btn.clicked.connect(self.toggle_game)

        # 5. СТАТИСТИКА И ИСТОРИЯ
        self.stats_label = QLabel("Последний: --- | Средний: ---")
        self.stats_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.stats_label.setStyleSheet(
            "font-size: 18px; color: white; font-weight: bold; background: #222; padding:
10px; border-radius: 5px;")

        # Блок истории (Заголовок, кнопка очистки и список)
        history_layout = QVBoxLayout()
        history_header = QHBoxLayout()
        history_title = QLabel("ИСТОРИЯ ПОПЫТОК:")
        history_title.setStyleSheet("color: #2491b3; font-weight:bold; font-size: 14px")

        self.clear_btn = QPushButton("Очистить историю результатов")
        self.clear_btn.setFixedSize(200, 30)
        self.clear_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.clear_btn.setStyleSheet("""
                    QPushButton {
                        background-color: #333; color: #ff5555;
                        border-radius: 4px; font-size: 10px; font-weight: bold;
                    }
                    QPushButton:hover { border: 1px solid #ff5555; }
                """)
        self.clear_btn.clicked.connect(self.clear_history)

        history_header.addWidget(history_title)
        history_header.addStretch()
        history_header.addWidget(self.clear_btn)

        self.history_list = QListWidget()
        self.history_list.setFixedHeight(120)
        self.history_list.setStyleSheet(
            "background-color: #1a1a1a; color: #2491b3; border: 1px solid #333; border-
radius: 10px; padding: 5px;")

        history_layout.addLayout(history_header)
        history_layout.addWidget(self.history_list)
        # 6. ВЫХОД
        self.back_btn = QPushButton("В ГЛАВНОЕ МЕНЮ")
        self.back_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.back_btn.setFixedSize(220, 40)
        self.back_btn.setStyleSheet("""
                    QPushButton {background-color: #333; color: white; border-radius:
8px; font-weight: bold; }
                    QPushButton:hover { background-color: #444; border: 1px solid
#2491b3; }
                """)
        self.back_btn.clicked.connect(self.back_to_menu.emit)
        main_layout.addLayout(settings_layout)
        main_layout.addWidget(self.info_label)
        main_layout.addLayout(field_layout)
        main_layout.addWidget(self.action_btn)
        main_layout.addWidget(self.stats_label)
        main_layout.addLayout(history_layout)
        main_layout.addWidget(self.back_btn, alignment=Qt.AlignmentFlag.AlignCenter)

        self.setLayout(main_layout)

    def toggle_game(self):
        if not self.is_running:
            self.is_running = True
```

```
            self.action_btn.setText("СТОП")
            self.action_btn.setStyleSheet( "background-color: #ff5555; color: white;
font-weight: bold; border-radius: 5px; padding: 10px; font-size: 18px;" )
            self.start_new_attempt()
        else:
            self.stop_game()

    def stop_game(self):
        self.is_running = False
        self.is_active = False
        self.stimulus_timer.stop()
        self.next_round_timer.stop()
        self.action_btn.setText("СТАРТ ТРЕНИРОВКИ")
        self.action_btn.setStyleSheet("""
                QPushButton { background-color: #2491b3; color: black; border-radius:
5px; padding: 10px; font-size: 18px; font-weight: bold; }
                QPushButton:hover { background-color: #4aa0ba; }
        """)
        self.reset_zones()
        self.info_label.setText("Тест остановлен.")

    def start_new_attempt(self):
        if not self.is_running: return
        self.is_active = False
        self.reset_zones()
        self.info_label.setText("Приготовьтесь...")
        # Задержка перед вспышкой из настроек
        delay = random.randint(1000, self.interval_spin.value())
        self.stimulus_timer.start(delay)
        self.setFocus()

    def activate_stimulus(self):
        if not self.is_running: return
        self.is_active = True
        self.target_side = random.choice(["left", "right"])
        self.start_perf_time = time.perf_counter()
        if self.target_side == "left":
            self.left_zone.setStyleSheet("background-color: #2491b3; border: 4px solid
white; border-radius: 15px;")
            self.info_label.setText("<<< ЛЕВО")
        else:
            self.right_zone.setStyleSheet("background-color: #2491b3; border: 4px solid
white; border-radius: 15px;")
            self.info_label.setText("ПРАВО >>>")

    def keyPressEvent(self, event):
        if not self.is_running: return

        # Обработка фальстарта
        if not self.is_active:
            if event.key() in [Qt.Key.Key_Left, Qt.Key.Key_Right]:
                self.handle_error("ФАЛЬСТАРТ!")
            return

        reaction = (time.perf_counter() - self.start_perf_time) * 1000

        # Проверка правильности нажатия
        if event.key() == Qt.Key.Key_Left and self.target_side == "left":
            self.record_success(reaction, "Лево")
        elif event.key() == Qt.Key.Key_Right and self.target_side == "right":
            self.record_success(reaction, "Право")
        elif event.key() in [Qt.Key.Key_Left, Qt.Key.Key_Right]:
            self.handle_error("ОШИБКА СТОРОНЫ!")

    def record_success(self, reaction, side):
```

```
        self.is_active = False
        self.results.append(reaction)

        save_result("Сложный выбор", reaction)
        avg = sum(self.results) / len(self.results)

        self.stats_label.setText(f"Последний: {reaction:.1f} мс | Средний: {avg:.1f} мс")
        self.history_list.insertItem(0, f"✅ {side}: {reaction:.1f} мс")
        self.info_label.setText(f"Отлично! ({reaction:.0f} мс)")
        # Подсвечиваем зеленым на мгновение
        color = "#155a15"
        if side == "Лево":
            self.left_zone.setStyleSheet(f"background-color: {color}; border-radius:
15px;")
        else:
            self.right_zone.setStyleSheet(f"background-color: {color}; border-radius:
15px;")
        # Запускаем следующий раунд через 1 секунду
        self.next_round_timer.start(1000)

    def handle_error(self, msg):
        self.is_active = False
        self.stimulus_timer.stop()
        self.info_label.setText(msg)
        self.left_zone.setStyleSheet("background-color: #700; border: 2px solid red;
border-radius: 15px;")
        self.right_zone.setStyleSheet("background-color: #700; border: 2px solid red;
border-radius: 15px;")
        self.history_list.insertItem(0, f"❌ {msg}")
        # Пауза перед перезапуском после ошибки
        self.next_round_timer.start(1500)
    def reset_zones(self):
        self.left_zone.setStyleSheet("background-color: #1a1a1a; border: 2px solid #333;
border-radius: 15px;")
        self.right_zone.setStyleSheet("background-color: #1a1a1a; border: 2px solid #333;
border-radius: 15px;")
    def clear_history(self):
        self.results = []
        self.history_list.clear()
        self.stats_label.setText("Последний: --- | Средний: ---")
        self.info_label.setText("История очищена. Можно продолжать.")
        self.setFocus()
        clear_module_history("Реакция на сложный выбор")

    def load_history_from_file(self):
        """Загружает данные из файла при запуске приложения"""
        saved_data = load_results("Реакция на сложный выбор")

        for item in saved_data:
            reaction = item["reaction_ms"]
            date_str = item["date"]

            self.results.append(reaction)  # Добавляем в локальный массив для расчета
среднего
            self.history_list.insertItem(0, f"📄 {date_str} - {reaction:.1f} мс")

        # Если данные загрузились, обновляем табло статистики
        if self.results:
            avg = sum(self.results) / len(self.results)
            last = self.results[-1]
            self.stats_label.setText(f"Последний: {last:.1f} мс | Средний: {avg:.1f} мс")
```